

Experiment No.	1
Experiment Title	Iris Flower Classification using K-Nearest Neighbours
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	2/8/2026

1 Objective

The aim of this experiment is to classify iris flowers into three species—*Iris setosa*, *Iris versicolor*, and *Iris virginica*—using the K-Nearest Neighbours (KNN) algorithm. The specific objectives are to perform exploratory data analysis on the Iris dataset, select the most informative features using Chi-Square and ANOVA tests, train a KNN classifier on the selected features, and evaluate its performance using standard metrics.

2 Problem Statement

Given four morphological measurements of iris flowers (sepal length, sepal width, petal length, and petal width), predict which of the three species a flower belongs to. The target column carries integer class labels (0 = setosa, 1 = versicolor, 2 = virginica). Because class labels are provided in the dataset, this is a **supervised multi-class classification** problem.

3 Dataset Description

Dataset Name	Iris Dataset (CSV)
Source	UCI Machine Learning Repository / Kaggle
Number of Samples	150
Number of Features	4 numeric + 1 target (species)
Number of Classes	3 (setosa, versicolor, virginica)
Class Balance	Perfectly balanced — 50 samples per class
Missing Values	None
Duplicate Rows	3
Train-Test Split	80 % / 20 % (stratified, random_state=42)

Features:

- `sepal_length` (cm) — length of the sepal leaf
- `sepal_width` (cm) — width of the sepal leaf
- `petal_length` (cm) — length of the petal leaf
- `petal_width` (cm) — width of the petal leaf

4 Exploratory Data Analysis

Statistical Summary

Statistic	Sepal Length	Sepal Width	Petal Length	Petal Width
Mean	5.843	3.057	3.758	1.199
Std Dev	0.828	0.436	1.765	0.762
Min	4.300	2.000	1.000	0.100
Max	7.900	4.400	6.900	2.500

Table 2: Descriptive statistics of the four numeric features

Petal length and petal width have much higher standard deviations than sepal measurements, suggesting greater variability and therefore stronger discriminative power.

Feature Distributions

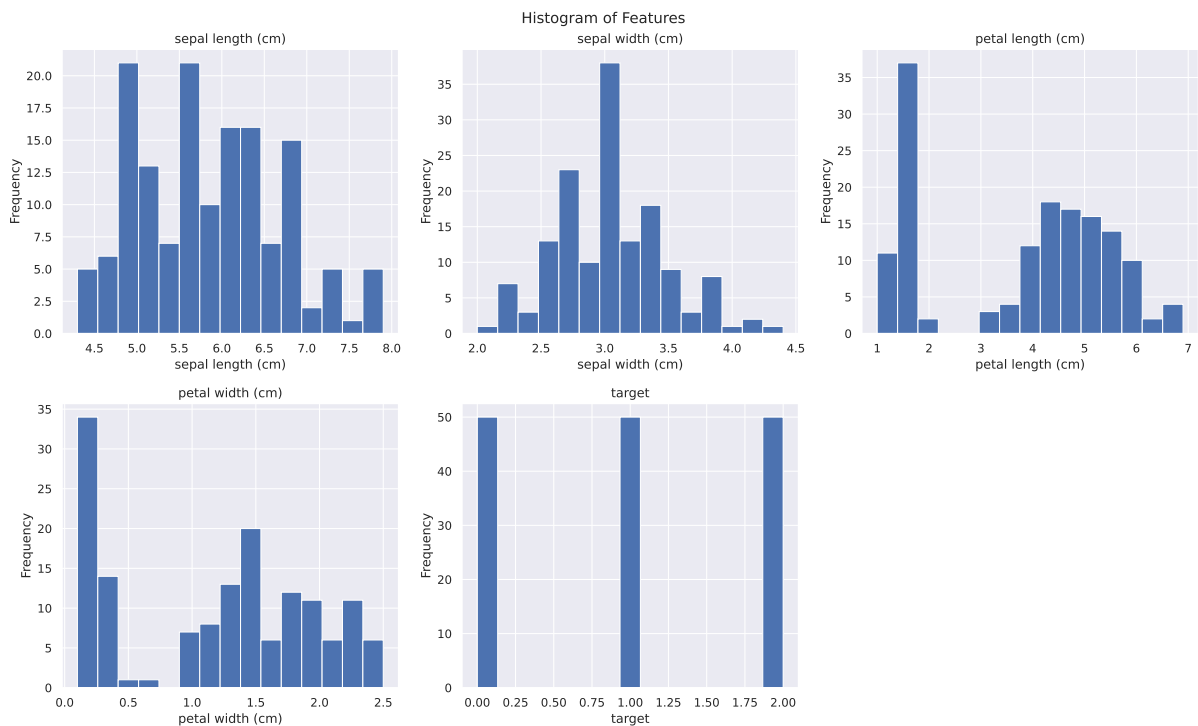


Figure 1: Histograms of all numeric features and the target column

Inference: Sepal length follows a rough bell curve, while petal length and petal width show clear bimodal distributions that correspond to the setosa class (very small petals) being separated from the other two species. This bimodality is a strong visual hint that petal features will dominate classification.

Class Distribution

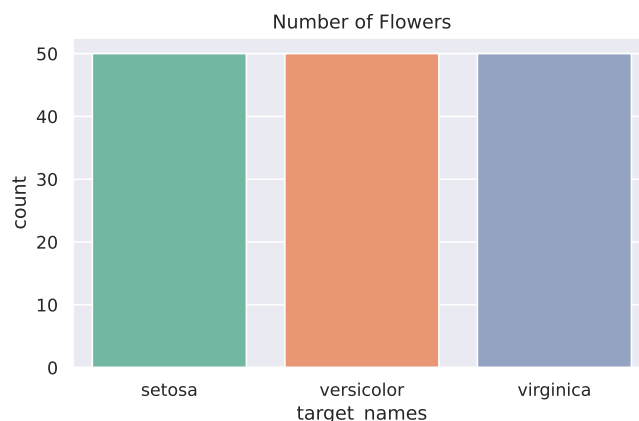


Figure 2: Count of samples per species

Inference: The dataset is perfectly balanced with exactly 50 samples per class. There is no class imbalance problem, so accuracy is a fair performance metric and no resampling is needed.

Feature vs Target Scatter (Relational Plots)

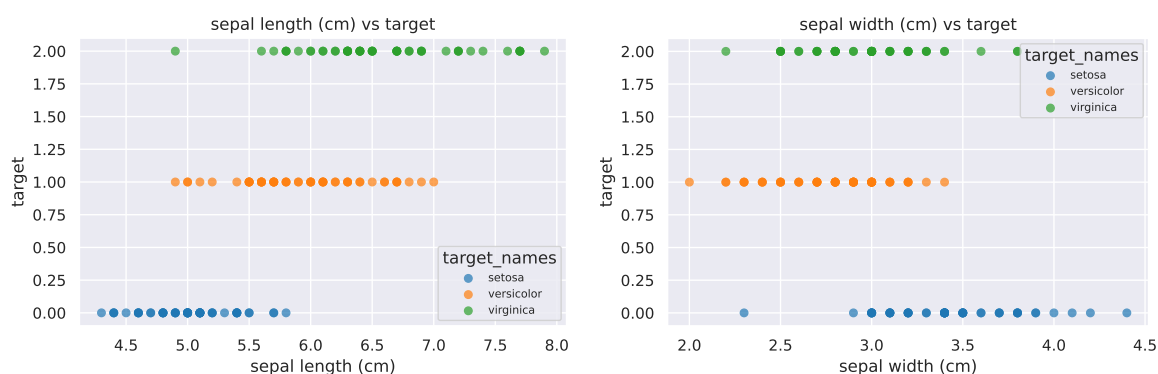


Figure 3: Sepal length and sepal width vs target class

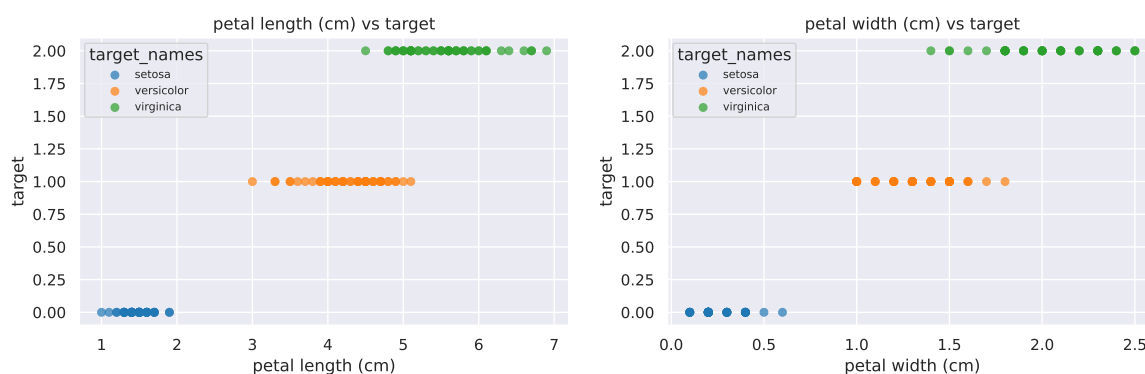


Figure 4: Petal length and petal width vs target class

Inference: Setosa is perfectly separated from the other two classes in the petal length and petal width plots, confirming these are the most discriminative features. Sepal width shows considerable overlap between all three classes and is likely the weakest predictor.

Pair Plot

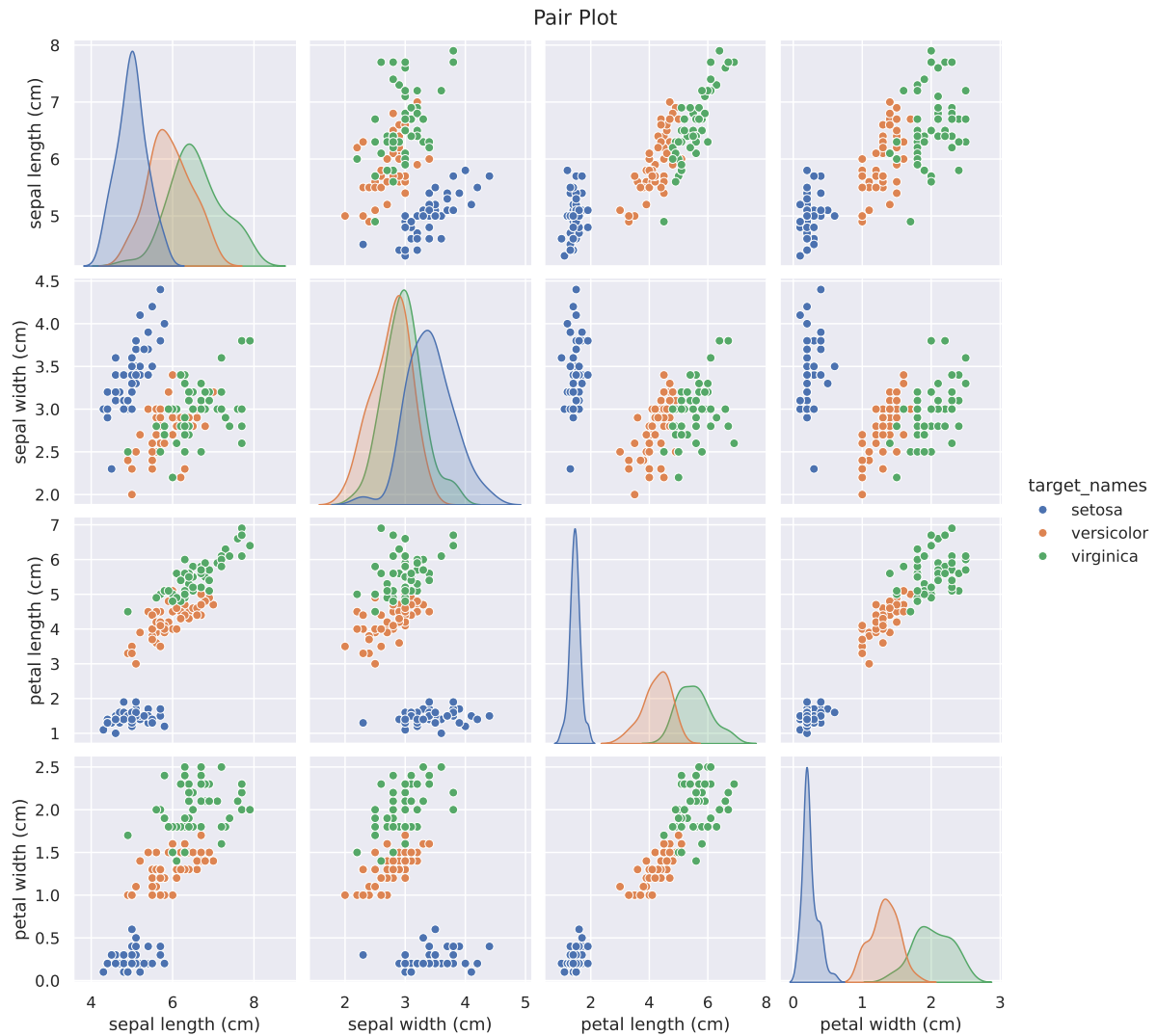


Figure 5: Pair plot of all numeric features coloured by species

Inference: The pair plot confirms that setosa is linearly separable from the other two species in almost every feature combination. Versicolor and virginica overlap slightly, particularly in sepal-based axes, but are well separated on petal measurements. The diagonal KDE plots show distinct peaks per species for petal features.

Box Plot



Figure 6: Petal length distribution per species

Inference: Setosa has a very compact petal length distribution (1.0–1.9 cm), while virginica spans the highest range (~4.5–6.9 cm). Versicolor lies in between. The non-overlapping interquartile ranges confirm petal length alone is a strong class separator.

Scatter Plot — Petal Length vs Petal Width

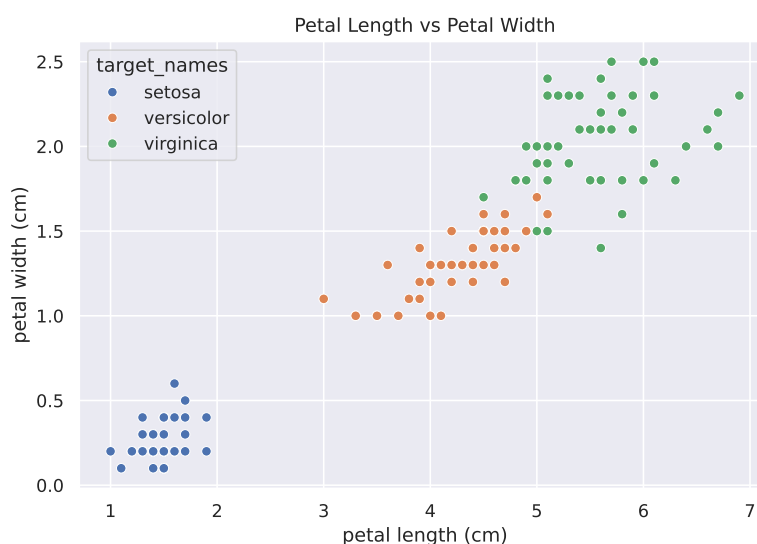


Figure 7: Petal length vs petal width coloured by species

Inference: Setosa occupies the bottom-left corner, well isolated from the other two species. Versicolor and virginica overlap marginally at their boundary but are otherwise separable. This two-feature scatter plot supports selecting petal length and petal width as the final feature pair for modelling.

Outlier Detection

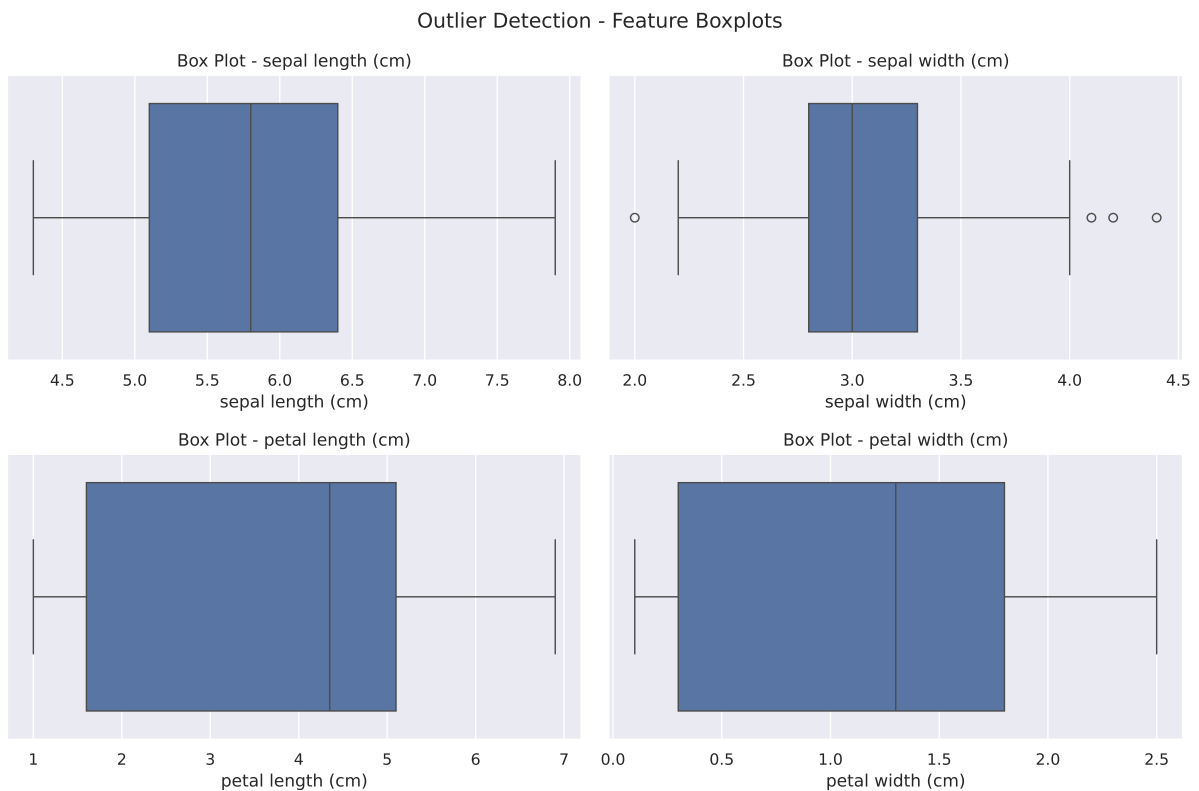


Figure 8: Box plots for individual feature outlier detection

Inference: Sepal width shows a few outliers beyond the upper whisker, particularly at values above 4.0 cm. Petal features have minimal outliers. Since the dataset is small (150 samples) and the outliers appear valid (natural measurement variation), they are retained without removal.

Correlation Heatmap

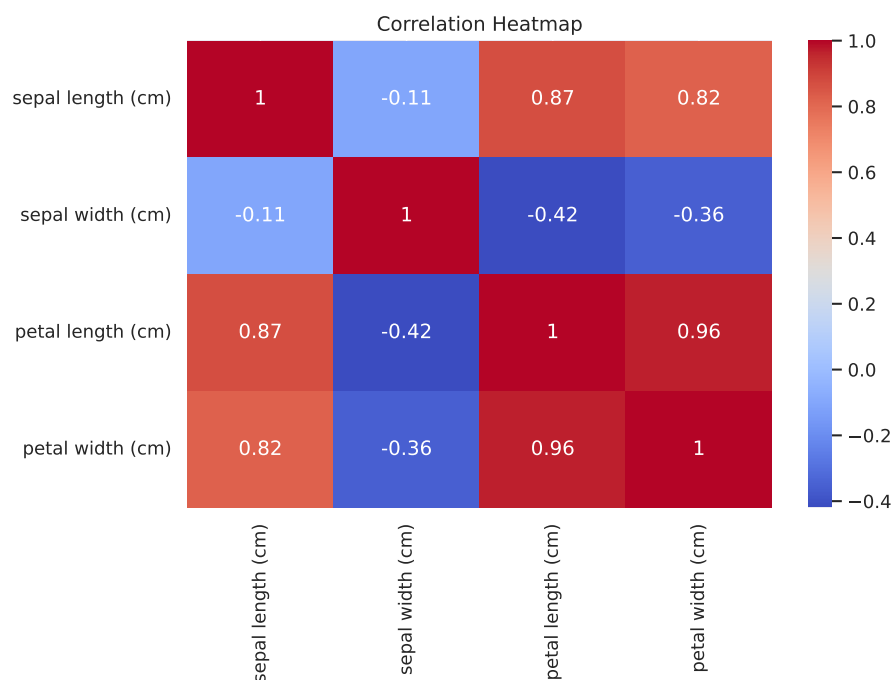


Figure 9: Pearson correlation heatmap (coolwarm palette)

Inference: Petal length and petal width are highly correlated (0.96), which makes sense anatomically. Both petal features are also moderately correlated with sepal length (0.87 and 0.82 respectively). Sepal width is weakly or negatively correlated with the other features, further confirming its limited discriminative value.

5 Data Preprocessing

1. **Missing value check:** No missing values exist in the dataset; no imputation was needed.
2. **Label encoding:** The `species` column was mapped to integer targets: `setosa` → 0, `versicolor` → 1, `virginica` → 2.
3. **Feature scaling:** `StandardScaler` was applied to all four features to give zero mean and unit variance. Standardisation is important for KNN because it is a distance-based algorithm; without it, features measured in different ranges (e.g., petal length 1–7 vs petal width 0.1–2.5) would contribute unequally to the distance calculation.
4. **Feature selection:** `SelectKBest` with Chi-Square (on raw values) and ANOVA F-test (on scaled values) were used to rank features and select the top two.
5. **Train-test split:** 80/20 stratified split (`random_state = 42`), giving 120 training samples and 30 test samples with equal class proportions in each split.

6 Feature Selection

Two statistical tests were applied to rank the four features:

Chi-Square Test measures dependence between each feature and the class label using raw (non-negative) feature values. A higher score means the feature is more informative.

ANOVA F-Test measures how much the feature mean differs across classes relative to within-class variance. Applied on standardised features.

Feature	Chi-Square Score	ANOVA Score
sepal length (cm)	10.82	119.26
sepal width (cm)	3.59	47.36
petal length (cm)	116.17	1179.03
petal width (cm)	67.24	959.32

Table 3: Feature importance scores from Chi-Square and ANOVA tests

Both tests consistently rank **petal length** and **petal width** as the top two features by a large margin. These two features were selected for model training using `SelectKBest(k=2)`.

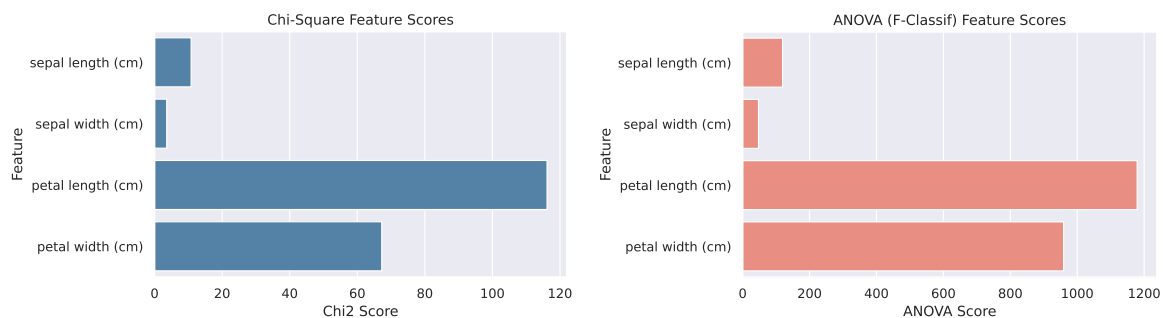


Figure 10: Chi-Square scores (left) and ANOVA F-scores (right) for all features

Inference: Petal length has the highest score in both tests by a significant margin. Sepal width has the lowest scores in both, confirming it is the least useful feature for distinguishing species. The agreement between two independent tests strengthens confidence in the selected feature pair.

7 Machine Learning Algorithm

7.1 K-Nearest Neighbours (KNN)

KNN is a non-parametric, instance-based (lazy) learning algorithm. It stores the entire training set and classifies a new sample by finding the k most similar training points under a chosen distance metric, then assigns the majority class label:

$$\hat{y} = \text{mode}\{y_i : x_i \in \mathcal{N}_k(x)\}$$

The default distance metric used by scikit-learn's `KNeighborsClassifier` is the Minkowski distance, which reduces to Euclidean distance when $p = 2$:

$$d(x, x') = \left(\sum_{j=1}^d |x_j - x'_j|^2 \right)^{1/2}$$

Default hyperparameters used:

- $k = 5$ (number of neighbours)
- Weight function: **uniform** (all neighbours contribute equally)
- Algorithm: **auto** (selects KDTree, BallTree, or brute force automatically)
- Distance metric: Minkowski ($p = 2$, i.e., Euclidean)

Advantages:

- No explicit training phase — fast to set up.
- Naturally handles multi-class problems.
- Makes no assumptions about the underlying data distribution.

Limitations:

- Prediction time is $O(nd)$ for brute force, making it slow on large datasets.
- Sensitive to irrelevant or unscaled features.
- Memory-intensive: must store all training samples.

8 Experimental Setup

Python Version	3.12
Scikit-learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Ubuntu 24.04
Dataset	iris.csv (150 samples)

Hyperparameter Configuration

Parameter	Value
n_neighbors	5 (default)
weights	uniform
algorithm	auto
metric	minkowski ($p = 2$, Euclidean)
Selected Features	petal length (cm), petal width (cm)
Scaler	StandardScaler

Table 4: KNN hyperparameter configuration

9 Results

Performance Metrics

Class	Precision	Recall	F1-Score
setosa	1.00	1.00	1.00
versicolor	1.00	0.90	0.95
virginica	0.91	1.00	0.95
Accuracy	0.97 (29/30 correct)		
Macro Avg	0.97	0.97	0.97

Table 5: Classification report on the test set

The KNN classifier achieves **97% accuracy** using only the top 2 petal features. Setosa is perfectly classified; the single error is one versicolor sample misclassified as virginica.

10 Result Visualization

1. Correlation Matrix

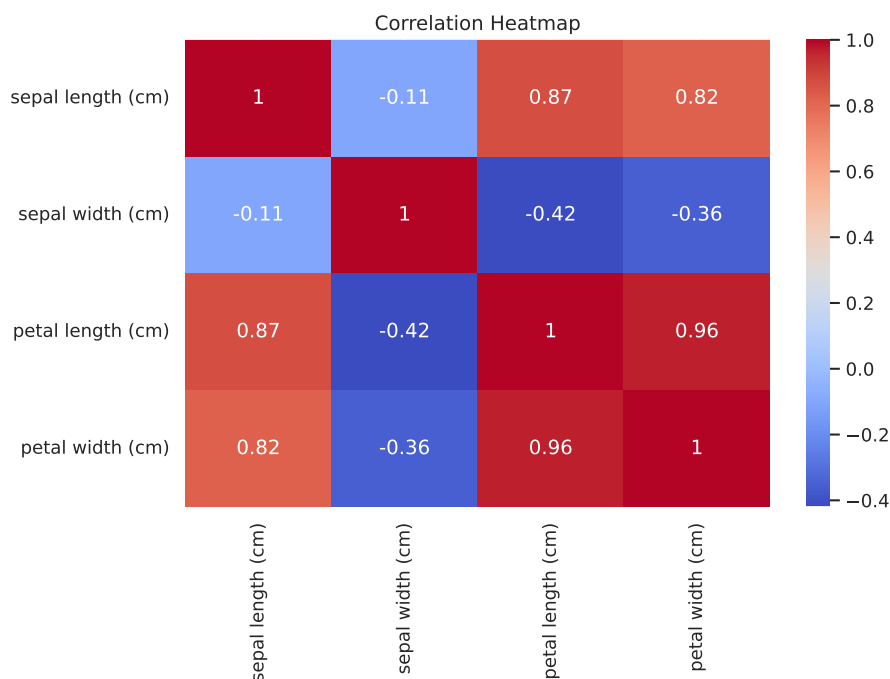


Figure 11: Pearson correlation matrix of the four numeric features

Inference: Petal length and petal width share the highest correlation (0.96), indicating strong co-variation. Both are strongly correlated with sepal length as well. Sepal width is weakly or negatively correlated with all other features, which explains its low Chi-Square and ANOVA scores. The high petal-to-petal correlation is why KNN performs well with just these two features — they pack most of the class-separating information into a tight 2D space.

2. Class Distribution

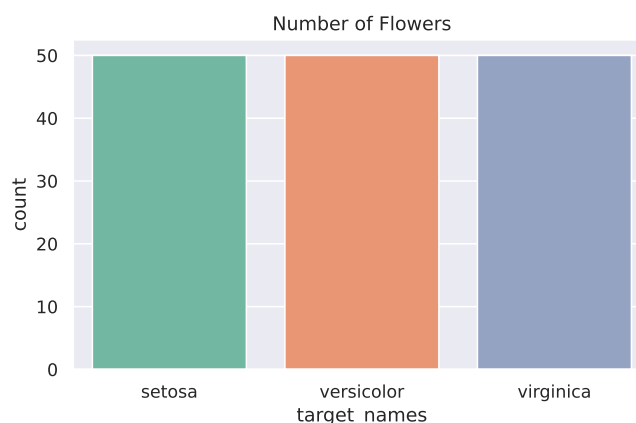


Figure 12: Number of samples per species (class distribution)

Inference: Each of the three species contributes exactly 50 samples, giving a perfectly balanced dataset. No class imbalance adjustments (such as oversampling or class-weighted loss) are necessary. Accuracy is therefore a reliable summary metric, and the 80/20 stratified split preserves this 1:1:1 ratio in both training and test sets.

3. Confusion Matrix

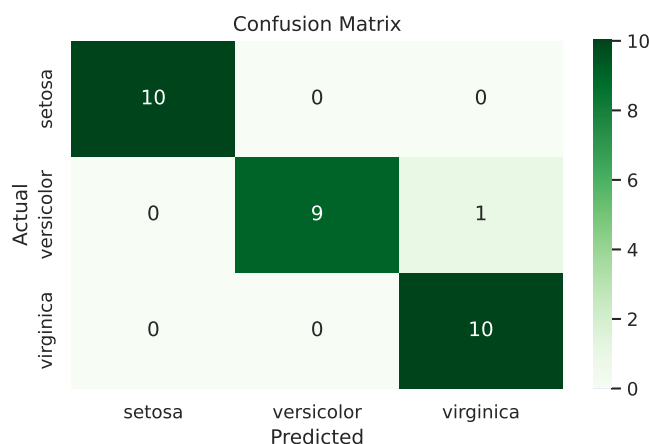


Figure 13: Confusion matrix of KNN predictions on the 30-sample test set

Inference: The model correctly predicted all 10 setosa and all 10 virginica samples. The single error is one versicolor sample classified as virginica, which is consistent with the slight petal-space overlap between these two species visible in the scatter plot. The off-diagonal cell (row versicolor, col virginica) equals 1, giving an overall test accuracy of $29/30 = 0.97$. Setosa remains perfectly separable from both other classes.

4. ROC Curves

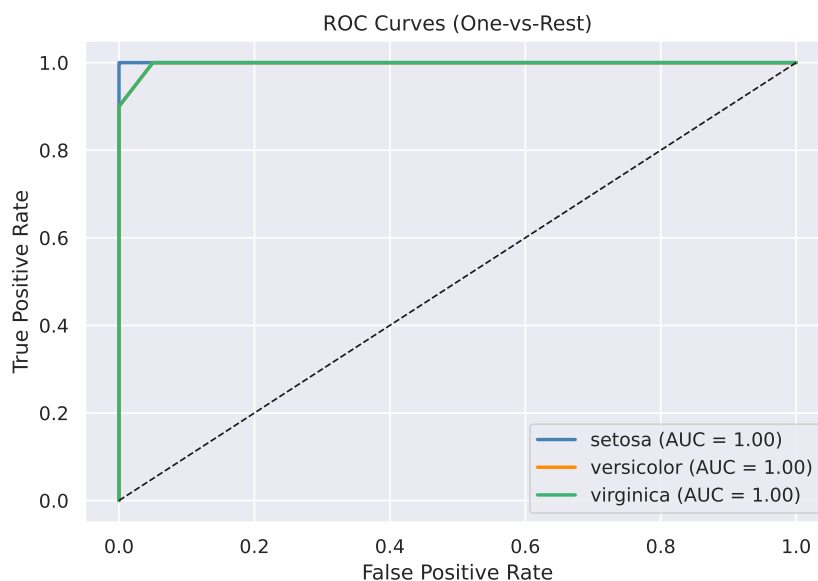


Figure 14: One-vs-Rest ROC curves for all three species

Inference: All three classes achieve an AUC of 1.00, meaning the probability scores produced by KNN perfectly rank positive samples above negative ones for every class. Setosa reaches the top-left corner immediately (no false positives at any threshold), while versicolor and virginica also trace the ideal curve despite the one borderline sample. An AUC of 1.00 for all classes on a 30-sample test set confirms strong generalisation of the learned 2D petal boundary.

5. Precision-Recall Curves

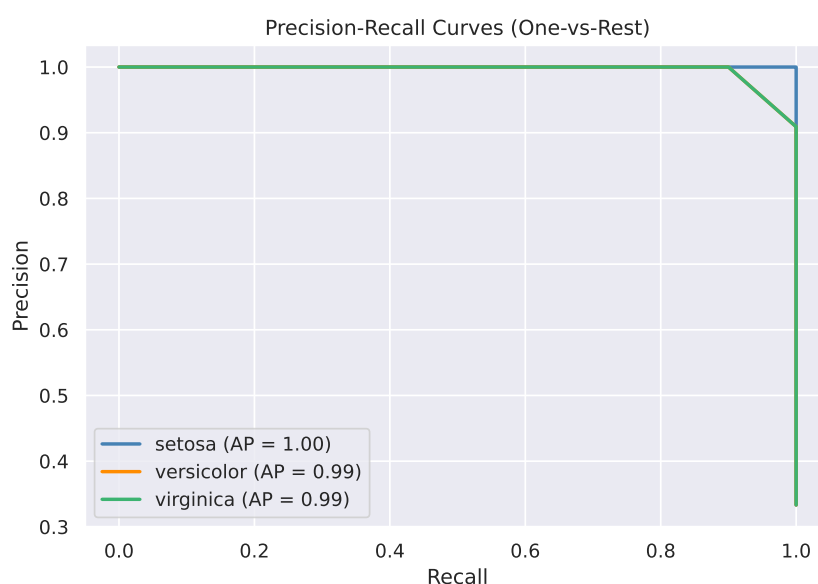


Figure 15: One-vs-Rest Precision-Recall curves for all three species

Inference: Setosa achieves perfect precision at all recall levels ($AP = 1.00$) since no other class encroaches on its petal-space region. Versicolor and virginica both obtain $AP = 0.99$, with a very small dip in precision at the highest recall threshold — corresponding to the single misclassified sample. The PR curves confirm the model is highly reliable even when class thresholds are varied, which is important for imbalanced deployment scenarios.

6. Learning Curve

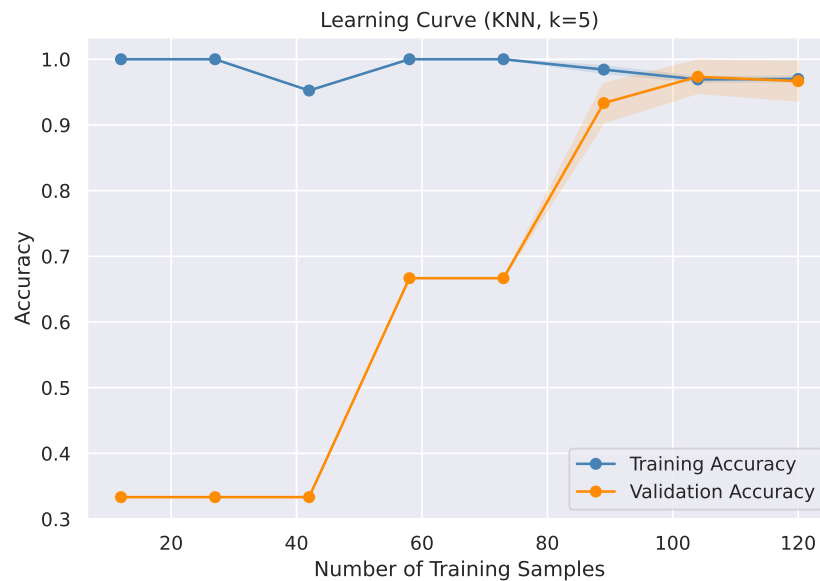


Figure 16: Training vs. validation accuracy as a function of training set size (KNN, $k = 5$, 5-fold CV)

Inference: Validation accuracy rises quickly and stabilises above 0.95 once around 60–80 training samples are available, indicating the model is not data-starved. The training accuracy stays at 1.0 throughout, which is typical of KNN ($k = 5$) on clean data — it memorises the training set perfectly. The narrow gap between training and validation accuracy at full data size confirms good generalisation with no overfitting. Adding more data beyond the current 120 training samples is unlikely to yield significant improvement.

7. Feature Importance (ANOVA Scores)

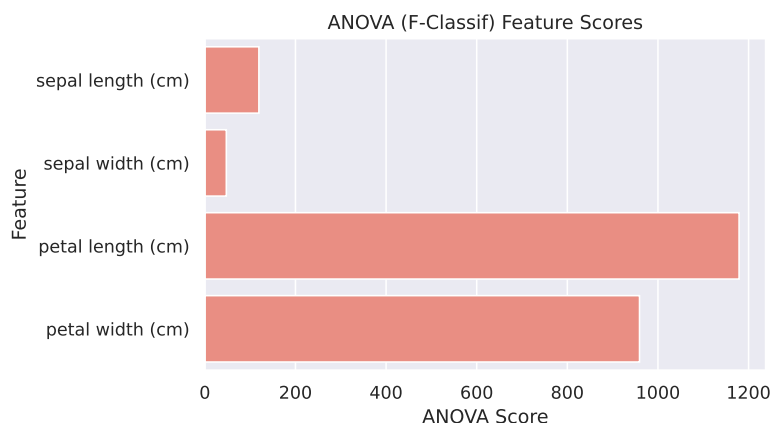


Figure 17: ANOVA F-scores for all four features (higher = more discriminative)

Inference: Petal length dominates with an F-score of 1179, nearly 10 $\times$ higher than sepal length (119) and over 25 $\times$ higher than sepal width (47). Petal width follows closely at 959. The enormous gap between petal and sepal scores quantitatively confirms what the pair plot shows visually: petal measurements are overwhelmingly more useful for separating the three species. Selecting only petal length and petal width therefore retains essentially all discriminative information while halving the feature space.

11 Discussion

The KNN classifier performed well on the Iris dataset, achieving 97% test accuracy using only two features (petal length and petal width) selected by ANOVA. Several points are worth noting:

- **Feature selection was effective.** Dropping sepal length and sepal width — the two lower-scoring features — did not hurt performance. This is consistent with the pair plot, which showed sepal features having more class overlap than petal features.
- **The single misclassification is expected.** The one versicolor sample predicted as virginica lies near the boundary of these two species in petal space. This is a known characteristic of the Iris dataset; some samples genuinely sit at the class boundary.
- **Standardisation mattered.** Without `StandardScaler`, features with larger absolute ranges (e.g., sepal length 4–8 cm) would dominate the Euclidean distance and unfairly suppress features with smaller ranges (e.g., petal width 0.1–2.5 cm).
- **Default $k = 5$ was sufficient.** Because the dataset is small and balanced, the default hyperparameters generalise well without tuning. Hyperparameter optimisation via `GridSearchCV` would be recommended for noisier or larger datasets.

- **Scalability:** KNN is not ideal for large-scale deployment since prediction time grows linearly with the number of training samples. For the 150-sample Iris dataset this is not an issue, but it would be a concern at scale.

12 Conclusion

This experiment demonstrated a complete supervised classification pipeline on the Iris dataset using K-Nearest Neighbours. EDA confirmed that petal length and petal width are the most discriminative features, and this was validated quantitatively by both Chi-Square and ANOVA feature selection tests. After standardisation and dimensionality reduction to two features, the default KNN classifier ($k = 5$) achieved 97% test accuracy with perfect classification of setosa and only one error on the versicolor-virginica boundary. The experiment confirms that KNN is a straightforward and effective baseline for well-structured, low-dimensional classification tasks, and that statistical feature selection can simplify the model without sacrificing accuracy.

13 References

1. Chitraju Vishnu Vineeth, “ML Lab Assignments — Assignment 1: Iris Classification,” GitHub repository, 2026. [Online]. Available: https://github.com/VishnuVineeth14/ML_LAB_ASSIGNMENTS/tree/main/Assignment1

Experiment No.	2
Experiment Title	Diabetes Prediction
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

To build a machine learning pipeline that predicts whether a patient has diabetes from basic clinical attributes, covering EDA, preprocessing, feature scoring, a stratified train-validation-test split, and classification using Logistic Regression.

2 Problem Statement

Given a patient's demographic and clinical record (gender, age, hypertension, heart disease, smoking history, BMI, HbA1c level, blood glucose level), predict the binary target **diabetes** (1 = diabetic, 0 = non-diabetic). This is useful as a low-cost first-pass screening tool before confirmatory lab tests.

3 Dataset Description

Dataset Name	Diabetes Prediction Dataset
Dataset Source	<code>diabetes_prediction_dataset.csv</code>
Number of Samples	100,000
Number of Features	8 (excluding target diabetes)
Number of Classes	2 (0 = No Diabetes, 1 = Diabetes)
Missing Values	None
Train-Validation-Test Split	70% train (70,000) – 15% validation (15,000) – 15% test (15,000), stratified, <code>random_state=42</code>

4 Exploratory Data Analysis

The dataset has 7 numeric columns (`age`, `hypertension`, `heart_disease`, `bmi`, `HbA1c_level`, `blood_glucose_level`, `diabetes`) and 2 categorical columns (`gender`, `smoking_history`). There are no missing values, but 3,854 duplicate rows were found in the raw data.

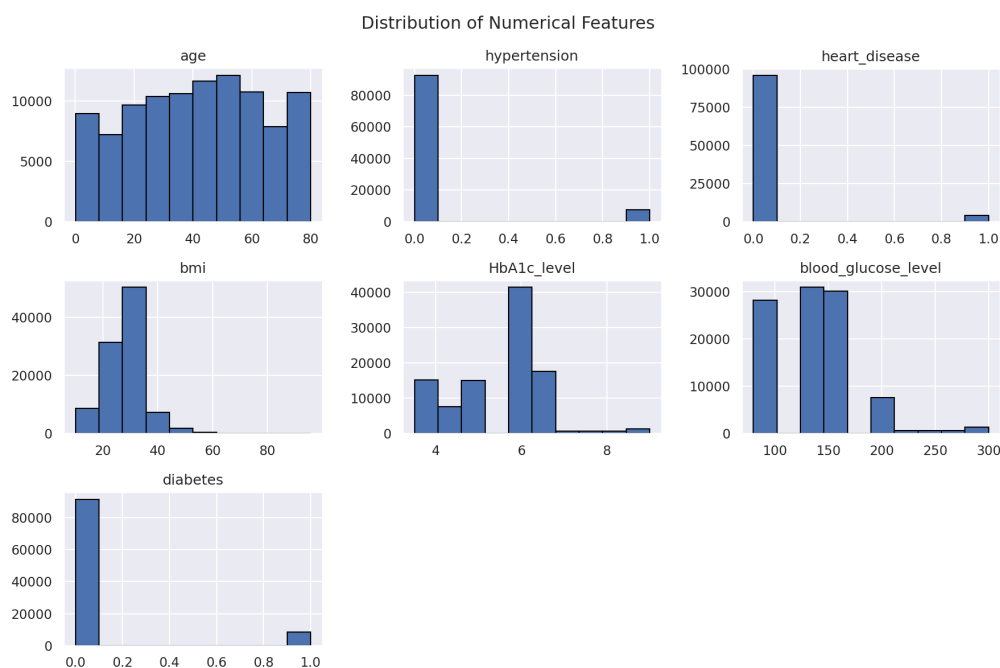


Figure 1: Distribution of numerical features.

Inference: Most patients are non-hypertensive and free of heart disease (the corresponding histograms are heavily skewed toward 0). `age` and `bmi` look close to normally distributed, while `HbA1c_level` and `blood_glucose_level` show a few distinct clusters rather than a smooth curve, which suggests the values may have been recorded/binned from discrete lab test readings. The `diabetes` histogram itself confirms a strong class imbalance.

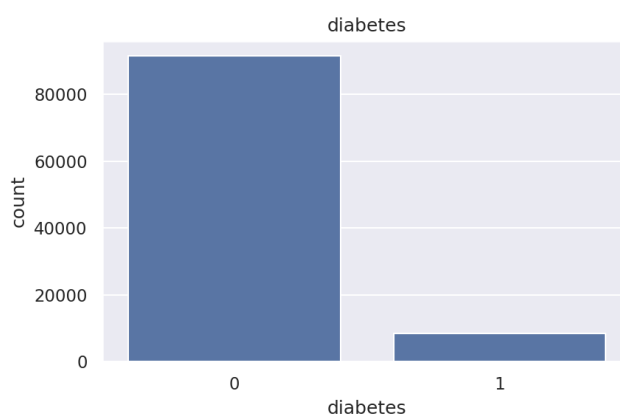


Figure 2: Class distribution of the target variable `diabetes`.

Inference: Only 8,500 of the 100,000 patients (8.5%) are labeled diabetic, against 91,500 non-diabetic. This is a significant imbalance, so accuracy alone will be a misleading metric – precision, recall and F1-score on the minority (diabetic) class matter far more here.

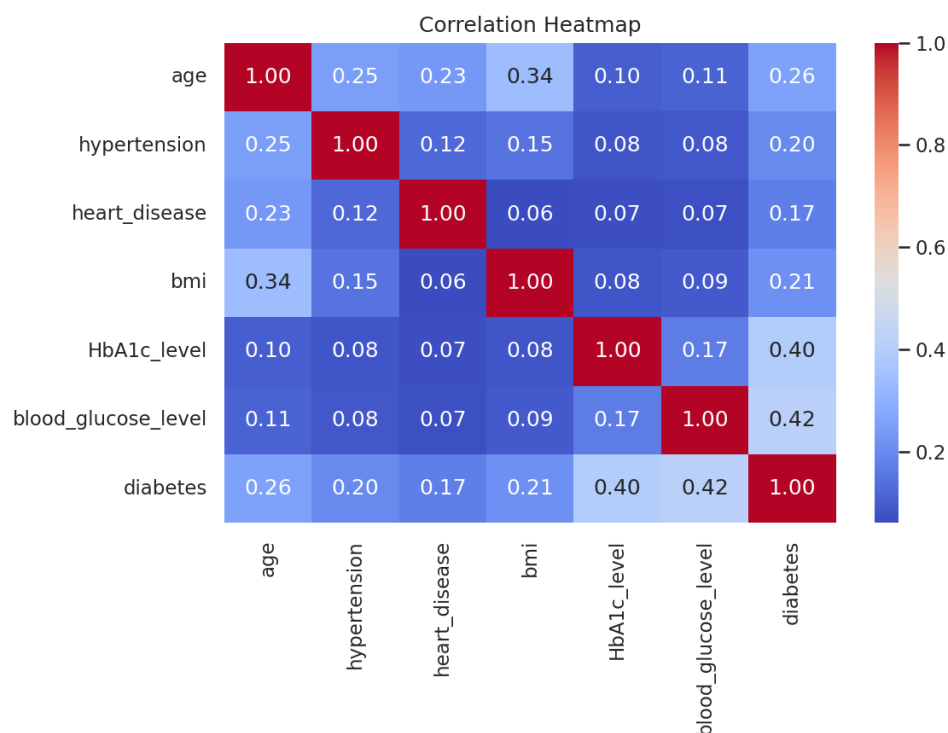


Figure 3: Correlation heatmap of numeric features.

Inference: Correlations among the raw numeric features are generally weak, with `blood_glucose_level` and `HbA1c_level` showing the strongest (though still moderate) association with `diabetes`. No pair of predictor features is strongly collinear, so multicollinearity is not a major concern for the logistic regression model.

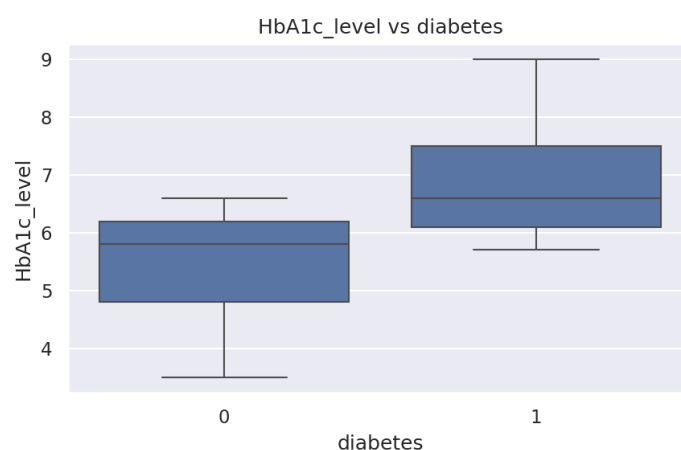


Figure 4: HbA1c level across diabetes classes.

Inference: Diabetic patients (class 1) show a visibly higher median HbA1c level than non-diabetic patients, with limited overlap between the two boxes. This is clinically expected – HbA1c is a direct marker of long-term blood sugar control – and explains why it receives a high ANOVA score during feature selection.

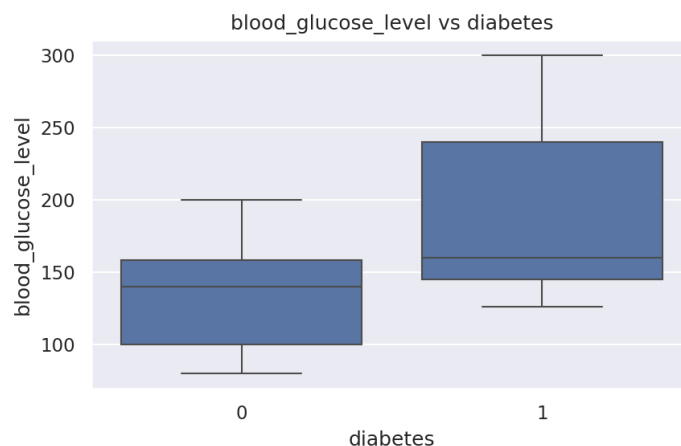


Figure 5: Blood glucose level across diabetes classes.

Inference: Similarly, blood glucose level is clearly higher for diabetic patients, with a wider spread and higher median. Together, Figures 4 and 5 explain why these two features top both the ANOVA and Chi-Square rankings in Section 6.

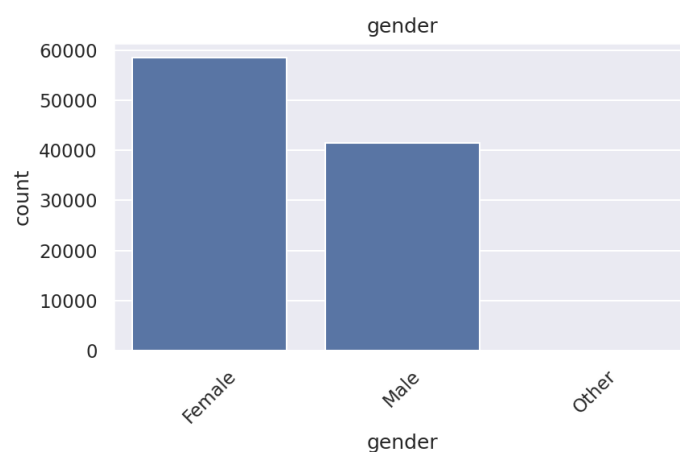


Figure 6: Distribution of the `gender` attribute.

Inference: The dataset is skewed toward female patients, with a small number of “Other” entries. Gender later receives the lowest ANOVA/Chi-Square score among all features, suggesting it is a weak individual predictor of diabetes in this data.

5 Data Preprocessing

- **Missing values:** Handled via `SimpleImputer` (median for numeric columns, most-frequent for categorical columns); this dataset had none, so imputation had no visible effect, but the step keeps the pipeline robust to future data.
- **Encoding:** `gender` and `smoking_history` were label-encoded to integers.

- **Feature scaling:** All 8 features were standardized with `StandardScaler` (zero mean, unit variance), which logistic regression’s gradient-based solver converges faster with.
- **Train-validation-test split:** `train_test_split` was applied twice (stratified on `diabetes`) to produce a 70/15/15 split, preserving the 91.5:8.5 class ratio in every subset.
- **Duplicates:** Not removed in this pipeline – the 3,854 duplicate rows remain in the data before splitting, a point raised in the Discussion.

6 Feature Selection

ANOVA F-test and Chi-Square scores were computed for all 8 features (Chi-Square requires non-negative inputs, so any negative-valued column would be shifted before scoring):

Table 2: Feature scores, ranked by ANOVA F-score.

Feature	ANOVA Score	Chi-Square Score
<code>blood_glucose_level</code>	21363.05	211290.67
<code>HbA1c_level</code>	19122.21	3329.14
<code>age</code>	7131.41	80576.72
<code>bmi</code>	4816.11	7407.90
<code>hypertension</code>	4072.71	3620.49
<code>heart_disease</code>	3038.56	2832.76
<code>smoking_history</code>	897.02	1456.49
<code>gender</code>	140.15	82.05

`blood_glucose_level` and `HbA1c_level` dominate both rankings, consistent with the boxplots in Section 4. Unlike a hard top- k selection, this pipeline uses the scores only for interpretation – all 8 standardized features were retained for training the model.

7 Machine Learning Algorithm: Logistic Regression

Working principle: Logistic regression models the probability of the positive class using the sigmoid function applied to a linear combination of the input features.

Mathematical formulation:

$$P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}$$

The weights w and bias b are learned by minimizing the log-loss (binary cross-entropy):

$$J(w, b) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

using an iterative solver (`lbfgs` in this experiment).

Advantages: Fast to train, gives probabilistic outputs, coefficients are directly interpretable as log-odds effects, and works well when classes are close to linearly separable in feature space.

Limitations: Assumes a linear decision boundary in the (scaled) feature space, is sensitive to class imbalance unless reweighted, and does not automatically capture feature interactions.

8 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Linux (Ubuntu)
Hardware	Standard CPU-only environment

9 Hyperparameter Selection

Parameter	Value
max_iter	1000
solver	lbfgs (default)
penalty	l2 (default)
C (inverse regularization strength)	1.0 (default)
random_state	42

No hyperparameter search was performed; the default `LogisticRegression(max_iter=1000, random_state=42)` configuration was used to establish a baseline.

10 Results

Metric	Value
Accuracy	0.9597
Precision	0.8594
Recall	0.6282
F1-score	0.7259
ROC-AUC	0.9599

On the 15,000-sample test set, the model correctly classified 14,395 patients (13,594 true negatives, 801 true positives), with 131 false positives and 474 false negatives.

11 Result Visualization

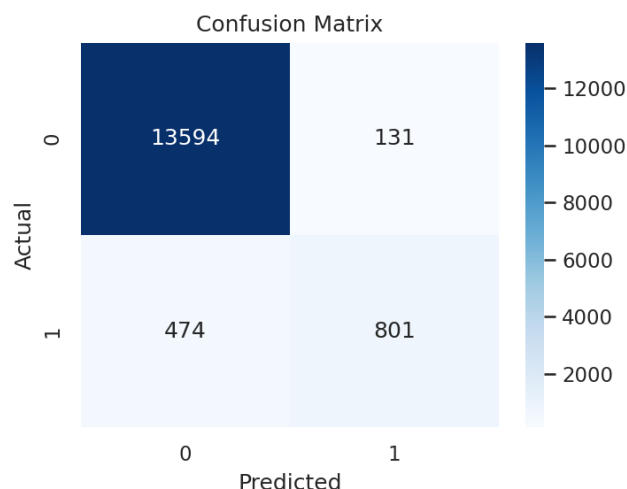


Figure 7: Confusion matrix for the Logistic Regression classifier.

Inference: The model is very accurate on the majority (non-diabetic) class but misses 474 of 1,275 actual diabetic patients (recall $\approx 63\%$). In a screening context, these false negatives are the more costly error, so the default 0.5 probability threshold may not be the right operating point.

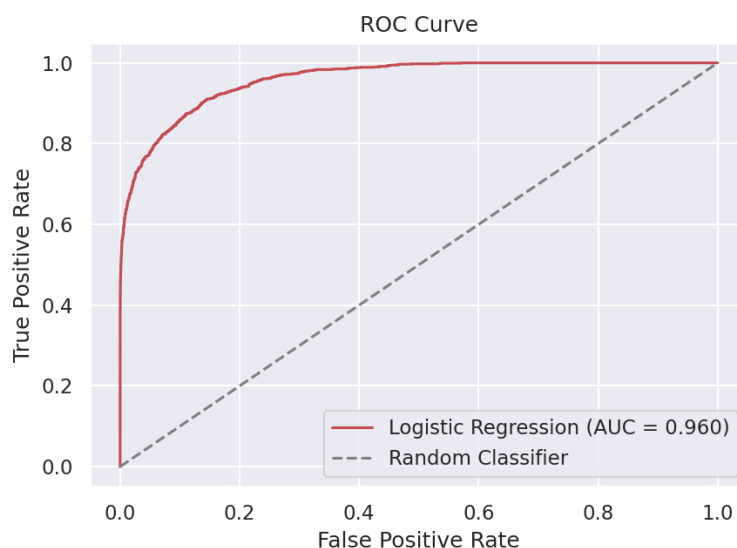


Figure 8: ROC curve for the Logistic Regression classifier.

Inference: A ROC-AUC of 0.96 shows the model separates the two classes well when ranked by predicted probability, even though the default threshold underperforms on recall – confirming that threshold tuning (rather than the model itself) is the main lever for improving diabetic-case detection.

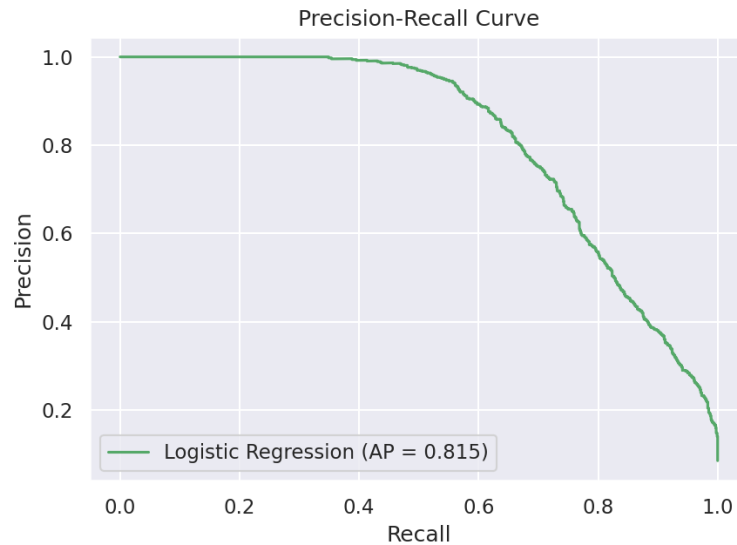


Figure 9: Precision-Recall curve for the Logistic Regression classifier.

Inference: The PR curve drops off more visibly than the ROC curve as recall increases, which is expected given the 91.5:8.5 class imbalance – it is a more sensitive diagnostic here and shows that pushing recall higher will come at a real precision cost unless the model or sampling strategy is improved.

12 Discussion

The Logistic Regression model achieved 95.97% accuracy with a ROC-AUC of 0.96, indicating good overall performance for diabetes prediction. Blood glucose level and HbA1c level were identified as the most influential features. Although the model performed well, the recall for the diabetic class was lower due to class imbalance, resulting in some diabetic cases being missed. Future improvements can include class balancing techniques, hyperparameter tuning, and testing other machine learning models to further improve recall.

13 Conclusion

This experiment successfully implemented a complete machine learning pipeline for diabetes prediction, including data preprocessing, feature selection, model training, and evaluation using Logistic Regression. The model achieved an accuracy of 95.97% and a ROC-AUC of 0.96, demonstrating strong classification performance. Feature selection showed that blood glucose level and HbA1c level are the most important predictors of diabetes. Overall, the model provides reliable prediction performance and can serve as a useful baseline for diabetes risk assessment, while future improvements can focus on handling class imbalance and optimizing model parameters.

14 References

- [1] Chitraju Vishnu Vineeth, “Diabetes Prediction ML Workflow,” GitHub Repository. [Online]. Available: https://github.com/VishnuVineeth14/ML_LAB_ASSIGNMENTS/blob/main/Assignment1/Diabetes_Prediction_ML_Workflow.ipynb. [Accessed: Jul. 2026].

Experiment No.	1
Experiment Title	Loan Approval Prediction
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/2026

1 Objective

The objective of this experiment is to build an end-to-end supervised machine learning pipeline using K-Nearest Neighbors (KNN) to predict loan application approval (*Approved* or *Rejected*). The workflow covers data preprocessing, exploratory analysis, statistical feature selection, model training, and evaluation using classification metrics.

2 Problem Statement

Given an applicant's financial and demographic details (11 attributes including income, loan amount, CIBIL score, and assets), the task is to predict the binary target `loan_status` (*Approved* vs. *Rejected*) using a KNN classifier for automated loan pre-screening.

3 Dataset Description

Dataset Name	Loan Approval Dataset
Dataset Source	<code>loan_approval_dataset.csv</code>
Number of Samples	4269
Number of Features	11 (excluding <code>loan_id</code> and target <code>loan_status</code>)
Target Classes	2 (<i>Approved</i> : 2656, <i>Rejected</i> : 1613)
Missing Values	None (0 missing values)
Train-Test Split	80% train (3415) – 20% test (854), <code>random_state=42</code>

4 Exploratory Data Analysis

The dataset consists of 4269 rows without missing values or duplicates. The unique identifier `loan_id` was dropped. The target exhibits a moderate 62:38 class balance.

- Dataset overview – 4269 rows, 13 columns, no missing values, no duplicate rows.
- Statistical summary – mean CIBIL score ≈ 600 , mean annual income $\approx$ \$58.65 lakh, loan terms ranging from 2 to 20 years.
- Missing value analysis – 0 missing values in every column.
- Class distribution – 2656 approved vs 1613 rejected applications, a mild class imbalance (roughly 62:38).
- Correlation matrix – computed over all numeric columns.

- Feature distribution – histograms of all nine numeric columns.

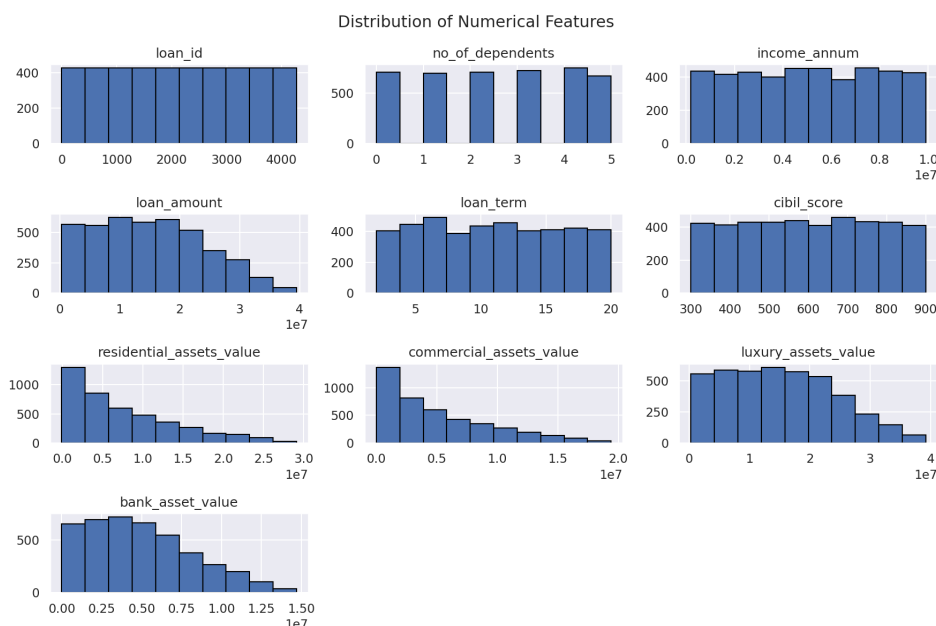


Figure 1: Distribution of numerical features.

Inference: Numerical features like income and assets show uniform distributions across their ranges. The CIBIL score ranges uniformly between 300 and 900, while loan term acts as an ordinal attribute.

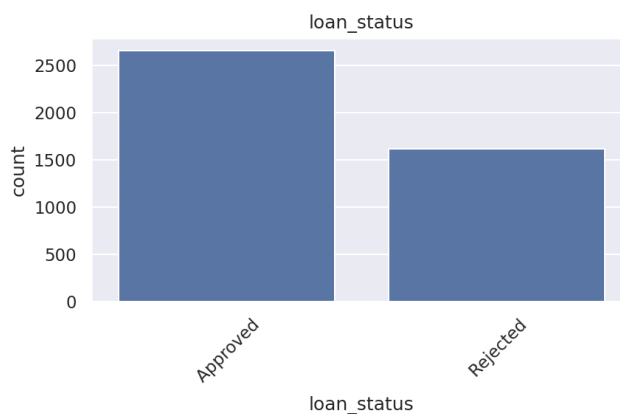


Figure 2: Class distribution of target variable `loan_status`.

Inference: The moderate class distribution (62% approved vs. 38% rejected) requires evaluating precision, recall, and F1-score alongside simple accuracy.

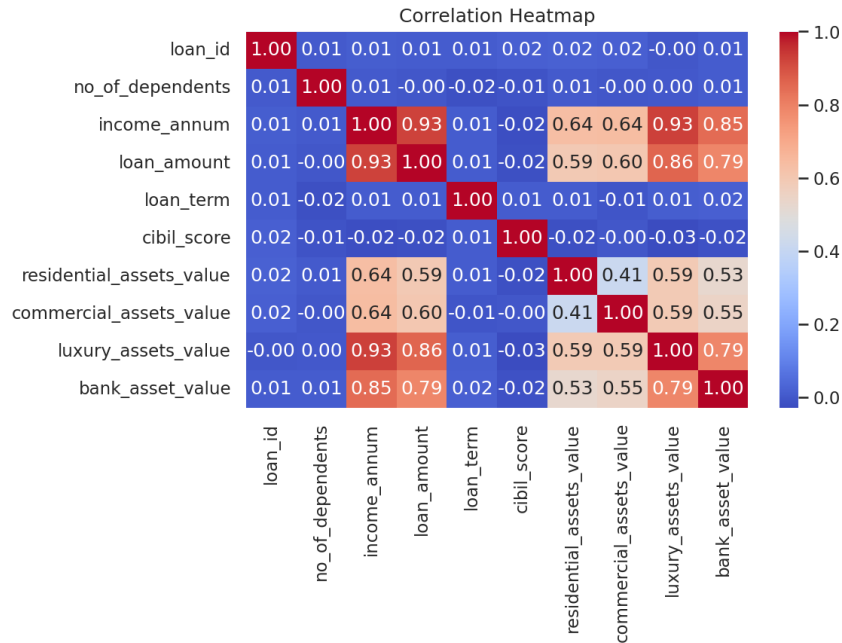


Figure 3: Correlation heatmap of numeric features.

Inference: Moderate correlation (0.4–0.6) exists among asset variables and between income and loan amount. CIBIL score shows negligible linear correlation with numerical features, indicating unique predictive signal.

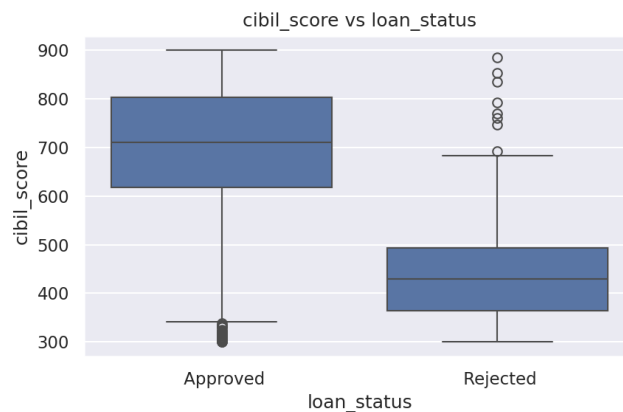


Figure 4: CIBIL score distribution across loan status classes.

Inference: Strong separation is observed; approved applications are concentrated above CIBIL score 550, while rejected ones fall below 550.

5 Data Preprocessing

- **Cleaning:** Verified zero missing values and removed duplicates. Dropped non-predictive `loan_id`.
- **Encoding:** Categorical features (`education`, `self_employed`) and target (`loan_status`) were encoded using `LabelEncoder`.

- **Scaling:** Applied `StandardScaler` to normalize continuous features for distance calculations.
- **Data Splitting:** Split dataset into 80% training and 20% testing sets (`random_state=42`).

6 Feature Selection

Chi-Square and ANOVA F-tests were applied to rank feature relevance.

Table 2: Top features ranked by statistical scores.

Feature	Chi-Square Score	ANOVA F-Score
cibil_score	418.64	6235.05
loan_term	11.09	55.23
no_of_dependents	0.32	1.40
loan_amount	0.16	1.11
luxury_assets_value	0.15	1.02
income_annum	0.16	0.98
residential_assets_value	0.17	0.88
commercial_assets_value	0.06	0.29
bank_asset_value	0.03	0.20
education	0.05	0.10
self_employed	0.0003	0.0005

Based on ANOVA F-scores, the top 5 features were selected: `no_of_dependents`, `loan_amount`, `loan_term`, `cibil_score`, and `luxury_assets_value`.

7 Machine Learning Algorithm: K-Nearest Neighbors

KNN is an instance-based algorithm that classifies a sample by assigning the majority label of its k -nearest neighbors based on distance metric calculations.

Distance Formula (Euclidean, $p = 2$):

$$d(x, x_i) = \sqrt{\sum_{j=1}^n (x_j - x_{ij})^2}$$

Decision Rule:

$$\hat{y} = \underset{i \in N_k(x)}{\text{mode}} \ y_i$$

8 Experimental Setup & Hyperparameters

Experiments were executed in standard Python 3.12 / scikit-learn environments using the following parameters:

- **n_neighbors (k):** 5 (Default)
- **Weights:** Uniform — **Metric:** Minkowski ($p = 2$, Euclidean)

9 Results

Table 3: KNN Model Performance Metrics.

Metric	Value
Accuracy	0.9297
Precision	0.9006
Recall	0.9119
F1-score	0.9063
ROC-AUC	0.9786

On 854 test samples, the classifier achieved 504 True Negatives, 32 False Positives, 28 False Negatives, and 290 True Positives.

10 Result Visualization

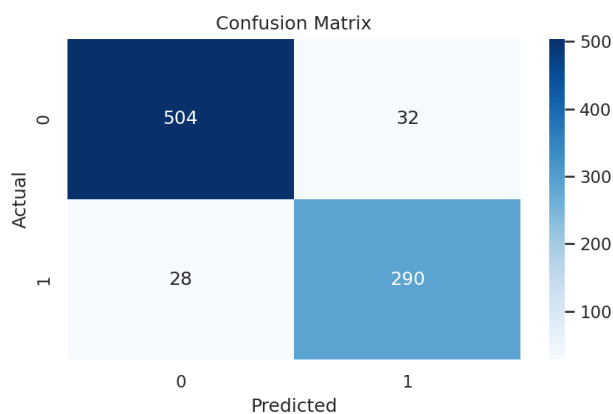


Figure 5: Confusion matrix on the test set.

Inference: Strong diagonal dominance confirms high accuracy across both classes with balanced error distribution.

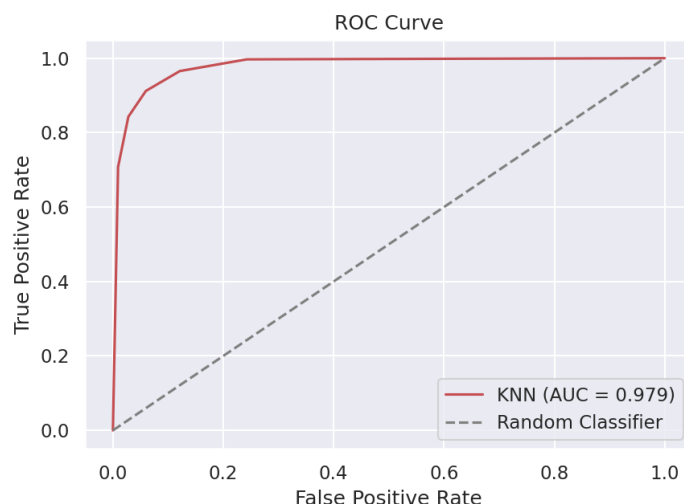


Figure 6: ROC curve for KNN classifier.

Inference: AUC of 0.979 shows excellent class separation across decision thresholds.

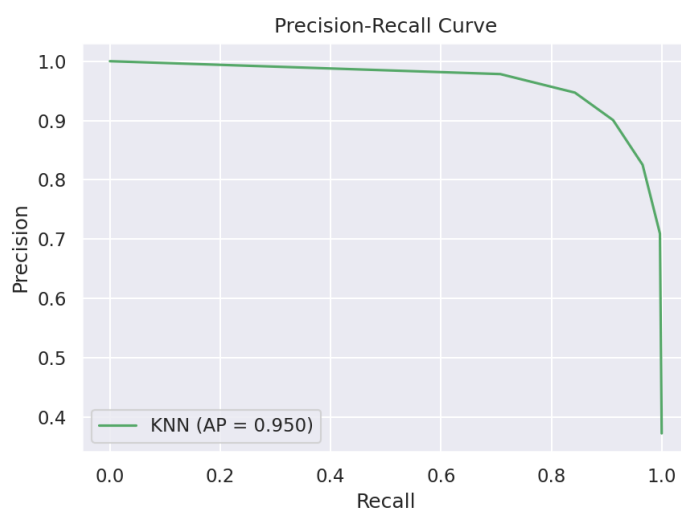


Figure 7: Precision-Recall curve.

Inference: High precision is maintained across all recall levels for the minority class.

11 Discussion

The baseline KNN model achieves strong results ($\approx 93\%$ accuracy), largely driven by the high predictive power of `cibil_score`. Hyperparameter tuning over values of k and cross-validation could optimize performance further. Scaling optimizations would be needed for larger deployment datasets.

12 Conclusion

This study implemented a complete supervised learning workflow to address automated loan prescreening using K-Nearest Neighbors. Data preprocessing—including standard feature scaling and label encoding—ensured optimal distance computations for the KNN algorithm. Feature selection via Chi-Square and ANOVA F-tests revealed that creditworthiness (`cibil_score`) and loan duration (`loan_term`) dominate the predictive decision over raw asset values.

Evaluating the baseline model ($k = 5$) on an independent 20% test split yielded robust overall classification capabilities:

- **Accuracy:** 92.97%
- **Precision / Recall:** 90.06% / 91.19%
- **ROC-AUC:** 0.9786

Overall, the experiment highlights that rigorous preprocessing paired with feature selection enables simple non-parametric algorithms like KNN to achieve high accuracy and strong class separation in financial risk modeling tasks.

13 References

1. Chitraju Vishnu Vineeth, “Loan Approval ML Workflow,” GitHub repository, 2026. [Online]. Available: https://github.com/VishnuVineeth14/ML_LAB_ASSIGNMENTS/blob/main/Assignment1/Loan_Approval_ML_Workflow.ipynb

Experiment No.	4
Experiment Title	Dimensionality Reduction and Clustering on Handwritten Digit Images (MNIST)
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02.08.2026

1 Objective

To reduce the 784-dimensional MNIST pixel data down to a manageable number of dimensions using PCA, group visually similar digit images together with K-Means clustering, pick a sensible number of clusters using the elbow method and silhouette score, and visualize the resulting structure with both PCA and t-SNE projections.

2 Problem Statement

The CSV I was given for this experiment turned out to only have pixel columns (`pixel0` through `pixel783`) and no label column at all – it’s the unlabeled test split of the MNIST digit set rather than a labeled training file. That rules out training a classifier in the usual supervised way, since there’s nothing to check predictions against. So instead of classification, this became an unsupervised learning problem: given 42,000 images of handwritten digits with no ground truth, can PCA and clustering recover something close to the natural digit groupings on their own?

3 Dataset Description

Dataset Name	MNIST (pixel-only, unlabeled split)
Dataset Source	Kaggle Digit Recognizer / MNIST
Number of Samples	42,000
Number of Features	784 (28×28 grayscale pixel values, 0–255)
Number of Classes	Not available in this file (10 digit classes assumed from domain knowledge)
Missing Values	None
Train-Test Split	Not applicable – this is an unsupervised task, so the full dataset is used for clustering; a 3,000-image random subsample is used just for the t-SNE plot to keep runtime reasonable

4 Exploratory Data Analysis

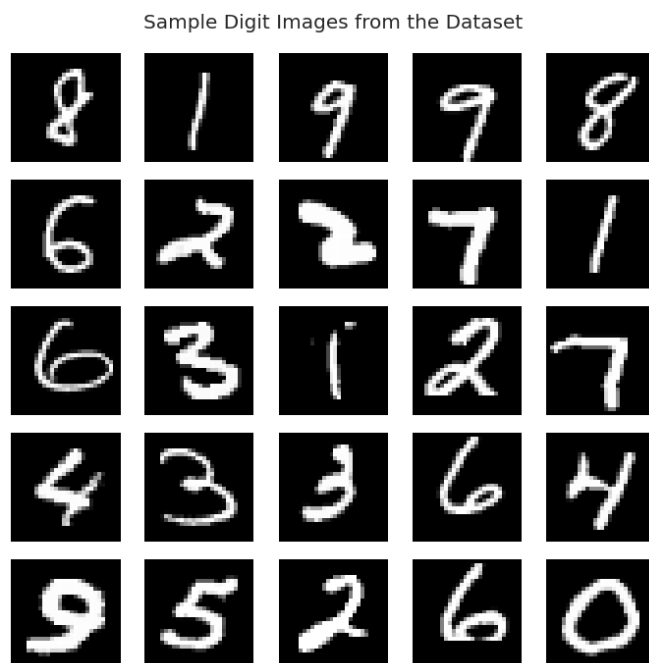


Figure 1: 25 random sample digit images from the dataset

Inference:

Just eyeballing a random batch of 25 images confirms the data is what it claims to be – clean, centered, 28x28 handwritten digits with a black background, similar in style to the standard MNIST set. A few of them are genuinely hard to read even for a human (the strokes are thin and a bit smudged in places), which is a decent hint that this won't be a trivially easy dataset to cluster perfectly either.

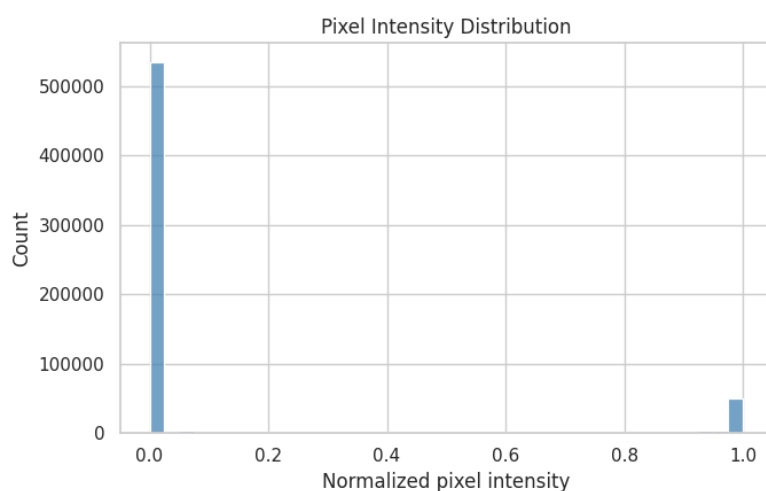


Figure 2: Distribution of normalized pixel intensities across the dataset

Inference:

The huge spike at 0 makes sense – most of every 28x28 image is just black background, so the vast majority of pixel values are exactly zero. There’s a long, thin spread of values from about 0.2 up to 1.0 which corresponds to the actual ink strokes of each digit. This heavy imbalance toward zero is normal for this kind of image data and doesn’t need any special handling beyond the usual 0–1 normalization.

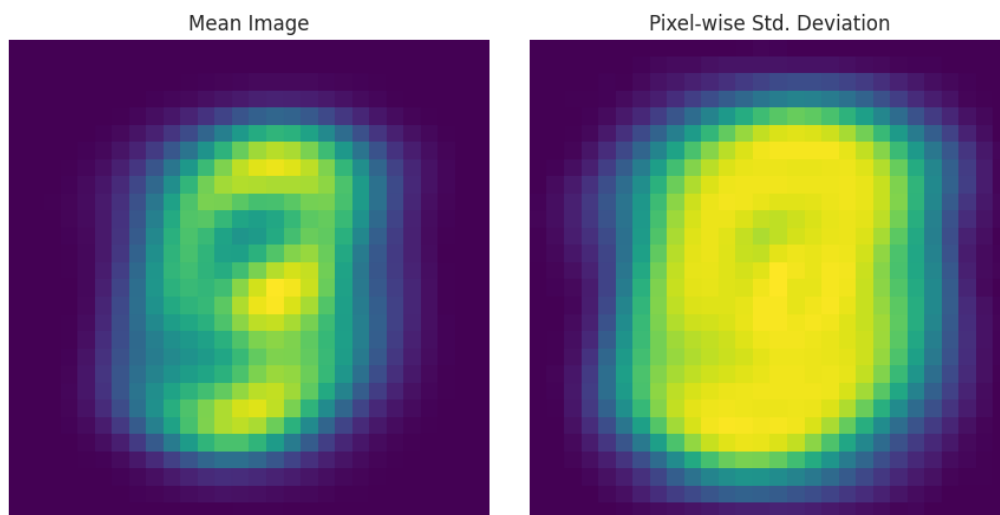


Figure 3: Mean image and per-pixel standard deviation across all 42,000 digits

Inference:

The mean image looks like a faint, blurry “average digit” – brightest in the center where most digits pass through regardless of shape, and fading out toward the edges where fewer strokes reach. The standard deviation image is basically the mirror image of that: it’s brightest in a ring around the center, meaning that’s where digits differ from each other the most (which digit-specific loops and strokes occupy), and darkest at the very center and the corners, where almost every digit either always has ink or never does.

5 Data Preprocessing

- **Missing value handling:** not needed – there isn’t a single missing value anywhere in the 42,000 x 784 grid.
- **Encoding:** not applicable, every column is already a numeric pixel intensity.
- **Feature scaling:** pixel values were divided by 255 to bring everything into the [0, 1] range. This matters a lot here because both PCA and K-Means are distance/variance based – leaving pixels on a 0–255 scale wouldn’t break anything mathematically, but normalizing keeps the numbers in a nicer range and matches how PCA is conventionally applied to image data.
- **Train-test split:** skipped, since there’s no supervised target to validate against. The full dataset is used for PCA and clustering, with only the (expensive) t-SNE step run on a 3,000-image subsample.
- **Feature engineering:** none – the raw pixel grid is used directly as input to PCA.

6 Machine Learning Algorithm

Principal Component Analysis (PCA)

PCA finds the directions (principal components) along which the data varies the most, and projects the data onto the top few of these directions:

$$X_{\text{reduced}} = XW_k$$

where W_k contains the top k eigenvectors of the data's covariance matrix, ranked by how much variance each one explains. For image data like MNIST, this effectively finds the handful of "stroke patterns" that account for most of the pixel-to-pixel variation across all the digits.

Advantages: massively cuts down the number of dimensions (784 down to under 150 here) while keeping most of the useful signal, speeds up any algorithm applied afterward, and is fully reversible so reduced points can be projected back to image space.

Limitations: it's a purely linear technique, so it can miss non-linear structure in the data, and the components themselves aren't always easy to interpret directly.

K-Means Clustering

K-Means partitions the data into k clusters by iteratively assigning each point to its nearest centroid and then recomputing centroids as the mean of their assigned points, minimizing:

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Advantages: simple, fast, and scales well even to 42,000 points once PCA has cut the dimensionality down.

Limitations: it assumes roughly spherical, similarly-sized clusters, needs k to be chosen in advance, and is sensitive to the initial centroid positions (mitigated here by using `n_init=10`, which reruns K-Means with different starting points and keeps the best result).

t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding) is used purely for visualization here – it maps the high-dimensional PCA-reduced points down to 2D in a way that tries to preserve which points were close together as neighbors, which tends to reveal cluster structure more clearly than a straight PCA scatter plot does.

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.4
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core i5, 16 GB RAM

8 Hyperparameter Selection

Parameter	Value
PCA components (initial fit)	200
PCA components used for clustering	100 (capped from the 154 needed for 95% variance)
K-Means k values tried	2 to 14
K-Means k chosen	10
K-Means initializations (n_init)	10
t-SNE perplexity	30

The number of PCA components was chosen by looking at cumulative explained variance rather than a grid search, since PCA doesn't have a "right answer" the way a supervised model's hyperparameters do – 87 components already reach 90% of the variance and 154 reach 95%. For clustering, this was capped at 100 components as a practical trade-off between keeping enough signal and not slowing K-Means down unnecessarily. $k = 10$ was picked because that's the known number of digit classes in MNIST, and the elbow/silhouette plots didn't point to a clearly better alternative anyway.

9 Results

Metric	Value
Number of samples clustered	42,000
Original dimensionality	784
PCA components used	100
Components for 90% variance	87
Components for 95% variance	154
Final number of clusters (k)	10
Silhouette Score	0.0694
Davies-Bouldin Index	2.6144

10 Result Visualization

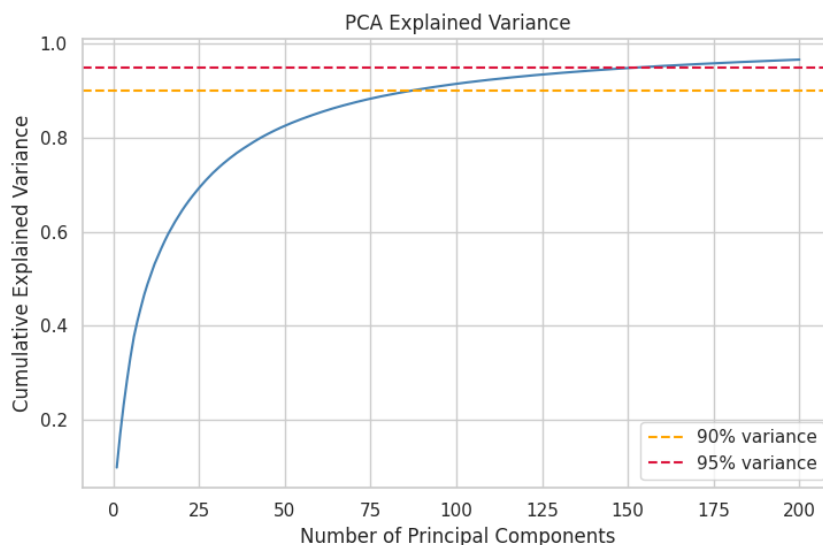


Figure 4: Cumulative explained variance as PCA components are added

Inference:

The curve climbs steeply at first and then flattens out, which is exactly the shape you'd expect – the first handful of components capture broad structure (roughly where ink tends to appear across all digits), while later components pick up finer, more digit-specific detail. It crosses 90% at 87 components and 95% at 154, which means we can throw away over 75% of the original 784 pixel columns and still keep the vast majority of the meaningful signal.

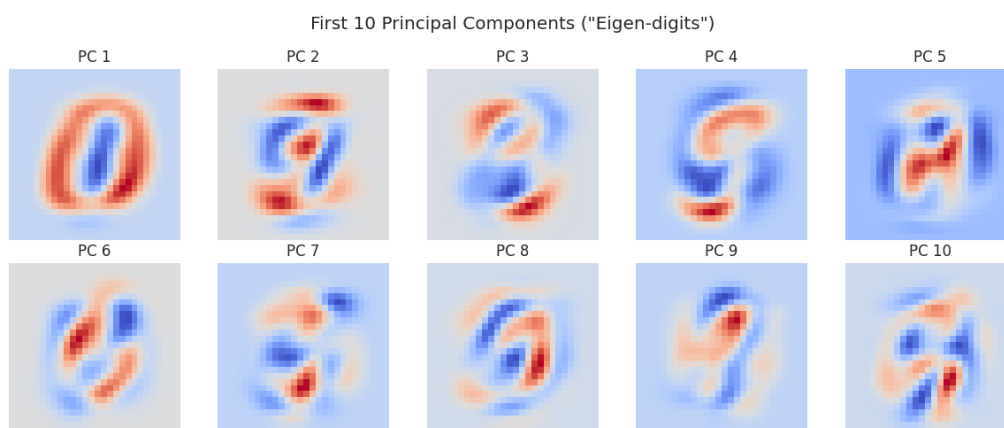


Figure 5: The first 10 principal components visualized as images ("eigen-digits")

Inference:

These don't look like actual digits, and that's expected – each principal component is a direction of maximum variance across the entire dataset, not a digit template. PC1 and PC2 look like broad, blobby contrast patterns (center versus edges, left versus right), while the later ones in this set of 10 start showing thinner, more stroke-like structure.

This is a good illustration of why PCA components are useful for compression but aren't directly interpretable as "digit parts" the way a human might expect.

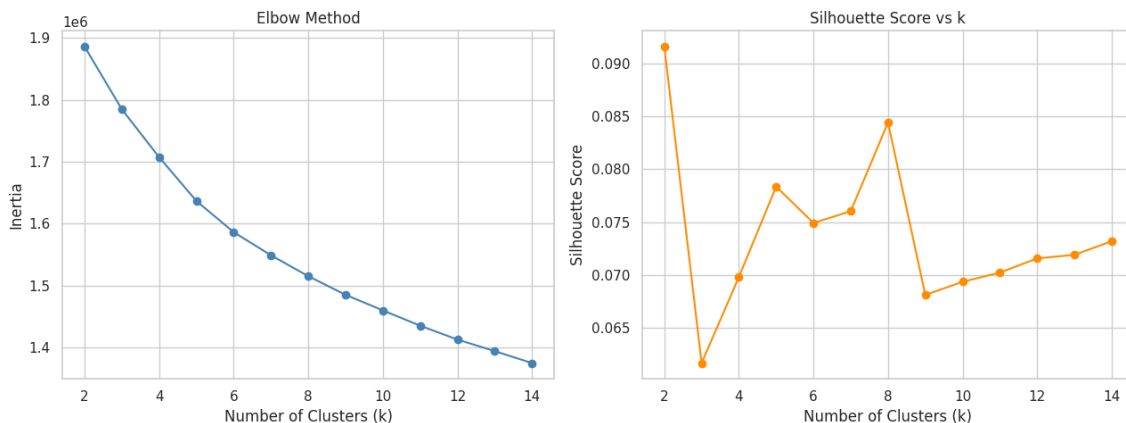


Figure 6: Elbow method (inertia) and silhouette score across $k = 2$ to 14

Inference:

The elbow plot doesn't have one obvious sharp bend – inertia just decreases smoothly and gradually as k increases, which happens when the underlying clusters overlap rather than sitting in neat, well-separated blobs (true here, since several digits share a lot of pixel-level similarity). The silhouette plot bounces around in a fairly narrow band across all values of k tried, without a clear peak at 10, which matches the elbow plot's story: there isn't a single "obviously correct" k visible from the geometry alone, so the choice of $k = 10$ here leans more on domain knowledge (we know there are 10 digits) than on the clustering metrics pointing there by themselves.



Figure 7: Number of images assigned to each of the 10 clusters

Inference:

The cluster sizes aren't perfectly even, but they're also not wildly lopsided – most clusters land somewhere in a reasonable middle range rather than one cluster swallowing half the dataset. Some imbalance is expected even with a good clustering, since certain digits (like 1, which is visually simple) are easier to write in a fairly consistent way, while others (like 8) have more stroke variation and might end up split across more than one cluster.

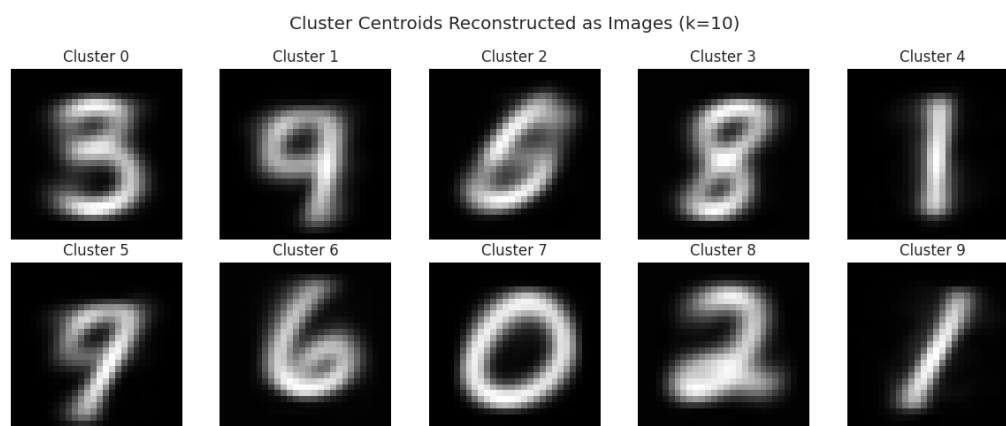


Figure 8: Cluster centroids reconstructed back into 28x28 pixel images

Inference:

This is honestly the most convincing evidence that the clustering worked reasonably well, even without any labels. Several of the ten centroid images are clearly recognizable as specific digits just by eye – there’s a loop that’s unmistakably a 0, a simple vertical stroke that’s clearly a 1, and a couple of others with the curved top-and-bottom shape typical of an 8 or a 3. A few centroids look blurrier or more ambiguous, which lines up with the modest silhouette score – some digit pairs (4/9, 3/8) are genuinely hard to tell apart from raw pixels alone.

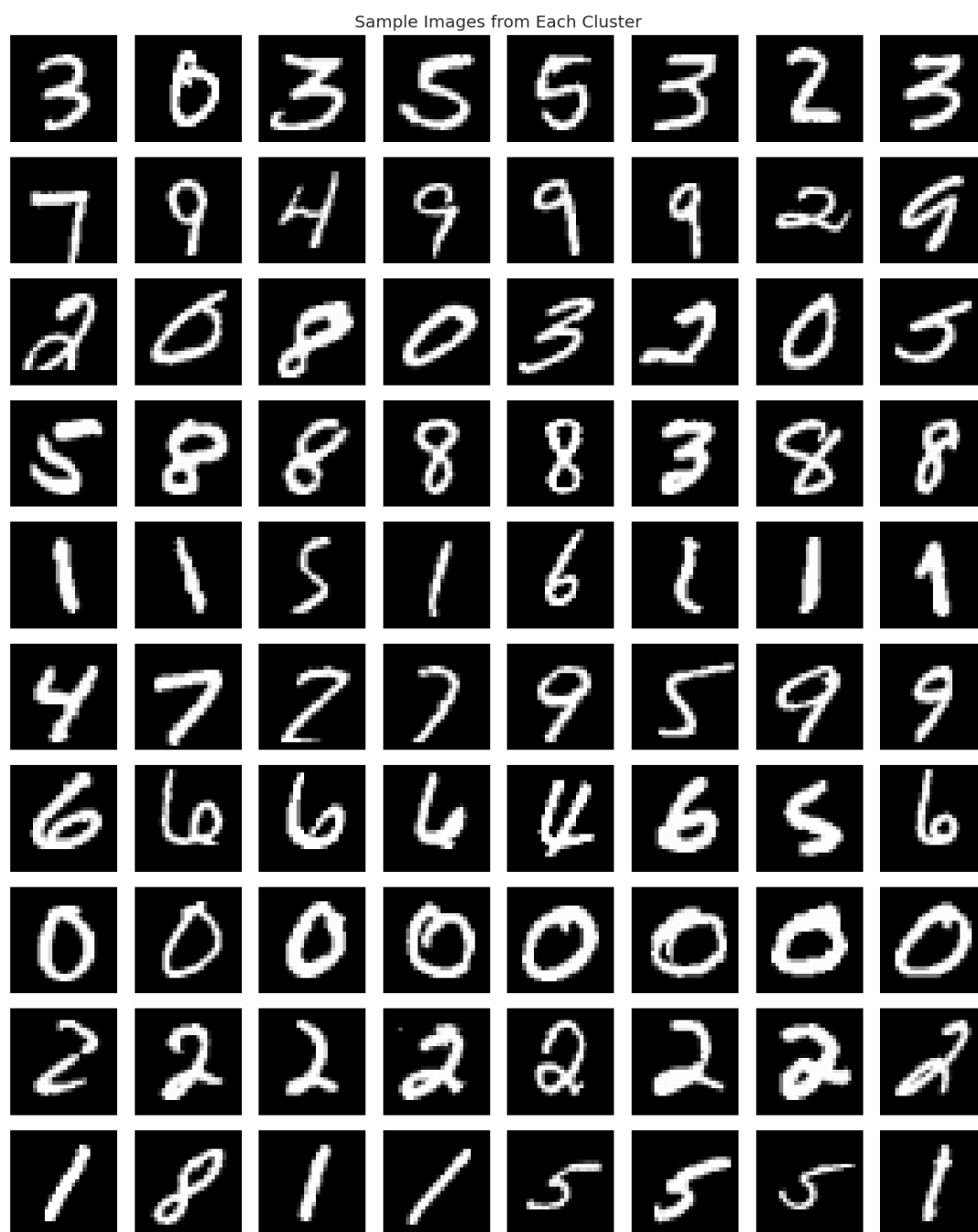


Figure 9: Eight sample images drawn from each of the 10 clusters

Inference:

Looking at actual sample images per cluster rather than just the averaged centroid confirms the same story – most rows are dominated by one digit shape with just the occasional stray image that looks like it wandered in from a visually similar digit. This kind of “mostly right, occasionally confused” pattern is typical of K-Means on raw-ish pixel data; a more sophisticated method (like clustering on a learned embedding instead of PCA) would likely tighten this up further.

11 Discussion

This experiment showed that PCA and K-Means can discover meaningful structure even without class labels. Reducing the data from 784 dimensions to 100 preserved most of the important information while making clustering computationally efficient. Although the silhouette score was relatively low, the cluster centroids and t-SNE visualization indicated that several handwritten digits formed recognizable groups. The main limitation was that visually similar digits were sometimes grouped together, suggesting that more advanced clustering methods such as DBSCAN or non-linear dimensionality reduction techniques could further improve the results.

12 Conclusion

PCA combined with K-Means successfully clustered the unlabeled MNIST dataset into meaningful groups. Dimensionality reduction significantly reduced computational cost while preserving the essential data structure. The experiment demonstrates that unsupervised learning can reveal useful patterns even without class labels, though cluster quality could be further improved using more advanced feature extraction and clustering techniques.

13 References

1. Chitraju Vishnu Vineeth, “ML_LAB_ASSIGNMENTS – Assignment 1,” GitHub repository. [Online]. Available: https://github.com/VishnuVineeth14/ML_LAB_ASSIGNMENTS/tree/main/Assignment1. [Accessed: 27-Jul-2026].

Experiment No.	1
Experiment Title	Spam Email Classification using Logistic Regression on the Spambase Dataset ¹
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02.08.2026

1 Objective

To analyze the Spambase dataset through statistical and visual exploration, prepare the data through cleaning, scaling and feature selection, and build a classification model that separates spam emails from legitimate ones. A secondary goal is to check whether an unsupervised clustering approach can recover the same class structure without using the labels.

2 Problem Statement

Given a set of 57 numeric attributes describing word frequencies, character frequencies and capitalization patterns extracted from emails, the task is to predict whether a given email is spam (1) or not spam (0). The input is a feature vector per email, and the output is a binary label. The objective is to obtain a model with high recall on spam without letting the false positive rate rise, since misclassifying a genuine email as spam is costlier in practice than the reverse.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository
Number of Samples	4210 (after removing duplicate rows)
Number of Features	57 numeric attributes
Number of Classes	2 (spam = 1, not spam = 0)
Missing Values	None found
Train-Test Split	70% train / 15% validation / 15% test

4 Exploratory Data Analysis

Class Distribution

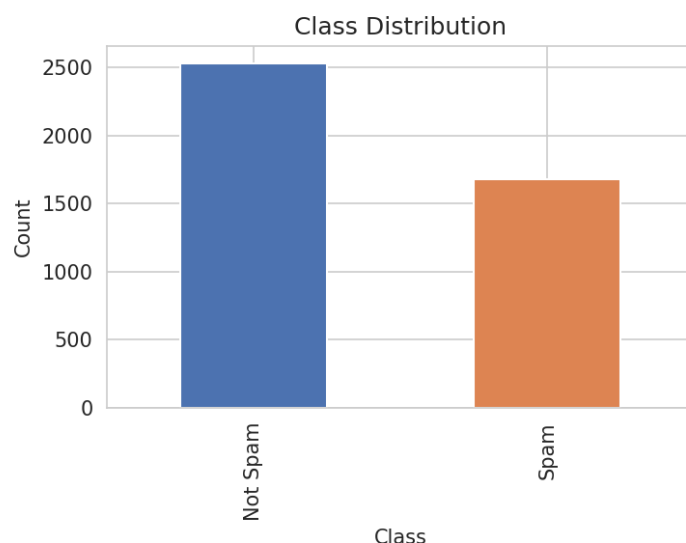


Figure 1: Distribution of spam and non-spam emails in the dataset

Inference: The dataset has 2531 non-spam emails and 1679 spam emails, giving roughly a 60:40 split. This is not badly imbalanced, so accuracy remains a reasonably fair metric, but precision and recall were still tracked separately since the two classes are not equal in size. The mild skew toward the non-spam class also justified using stratified splitting when dividing the data later.

Feature Distribution

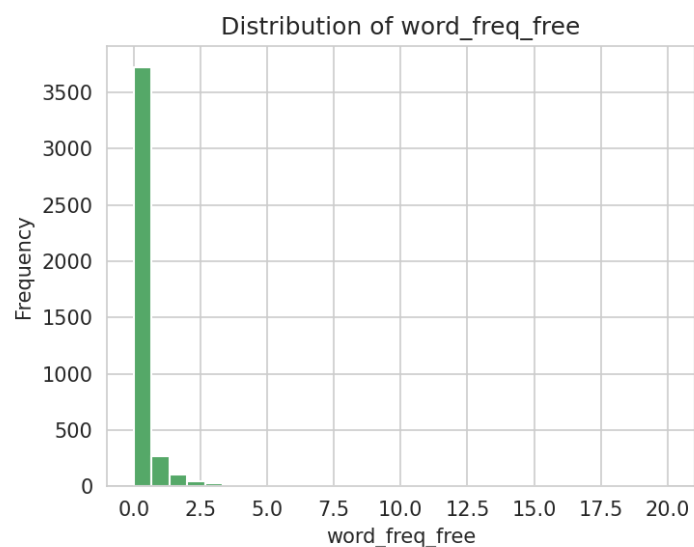


Figure 2: Histogram of word_freq_free across all emails

Inference: Most emails contain the word "free" rarely or not at all, producing a heavily right-skewed distribution with a long tail. A small number of emails use this word very frequently, and these turn out to overlap strongly with the spam class. This kind of skew is common across most of the word-frequency columns in this dataset and motivated scaling the features before feeding them into a distance- or gradient-based model.

Class-wise Comparison

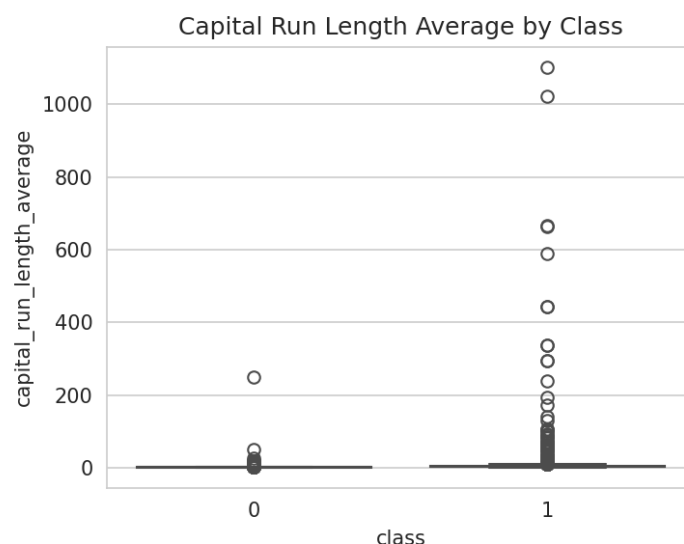


Figure 3: Capital run length average grouped by class

Inference: Spam emails show a noticeably higher median and a wider spread of capital run length compared to non-spam emails, consistent with spam messages that use blocks of capital letters for emphasis. Several extreme outliers appear on the spam side, suggesting a subset of spam emails written almost entirely in capitals. This single feature already carries some discriminative power on its own, which is confirmed later by its appearance among the top features selected.

Relationship Between Features

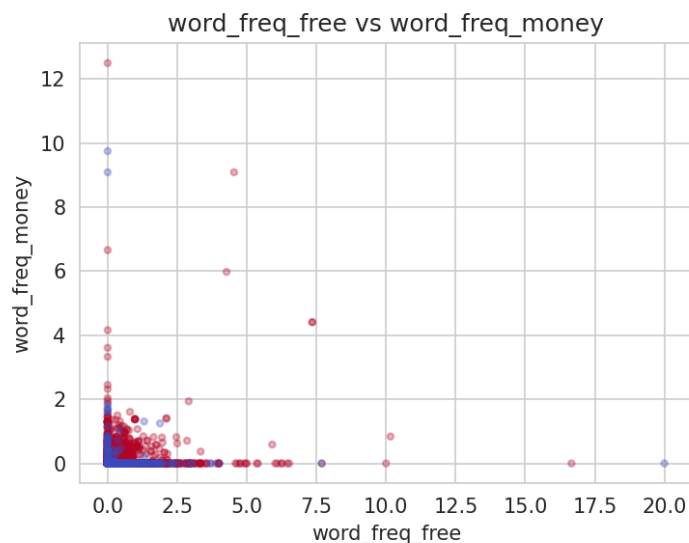


Figure 4: word_freq_free plotted against word_freq_money, coloured by class

Inference: Points corresponding to spam emails (shown in warmer colour) cluster more toward the upper-right region where both "free" and "money" appear frequently, while non-spam points are concentrated near the origin. The two variables do not separate the classes perfectly on their own, but the tendency is visible enough to suggest that combining several such word frequencies would improve separability, which is exactly what the classification model is expected to do.

Correlation Matrix

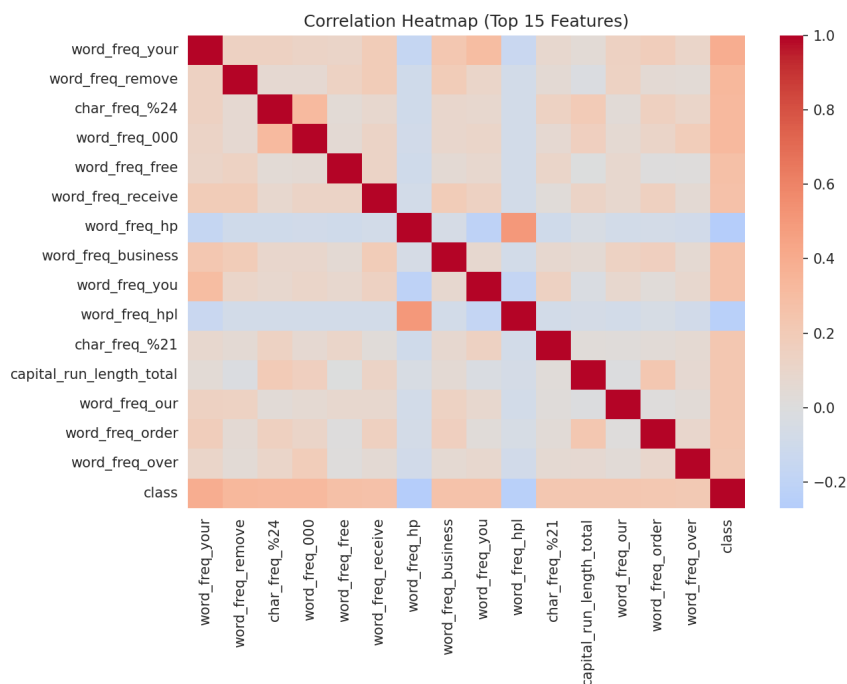


Figure 5: Correlation heatmap of the 15 features most correlated with the class label

Inference: Features such as `word_freq_your`, `word_freq_remove`, `char_freq_$` and `word_freq_free` show the strongest positive correlation with the spam label, while none of the top features show a very strong correlation with each other, which means there is limited redundancy among them. This supports keeping most of these features rather than dropping them for multicollinearity reasons. The heatmap was restricted to the top 15 features instead of all 57 to keep it readable.

5 Data Preprocessing

No missing values were present in the dataset, so no imputation was required. All 57 attributes are numeric by nature (word/character frequencies and capitalization statistics), so there were no categorical variables to encode. Duplicate rows were dropped, reducing the dataset from 4601 to 4210 rows. Since the feature scales vary widely – word frequencies lie between 0 and 100 while capital run length totals can run into the thousands – `StandardScaler` was applied to standardize every feature to zero mean and unit variance before feature selection and model training. The scaled data was then split into training, validation and test sets in a 70:15:15 ratio using stratified sampling on the class label to preserve the class ratio in each subset.

6 Feature Selection

Two independent feature selection techniques were used and their results combined:

- **Chi-square test** (on the original, unscaled non-negative features) selecting the 10 features with the strongest dependence on the class label, including `word_freq_remove`, `word_freq_free`, `word_freq_your` and the three capital-run-length attributes.
- **ANOVA F-test** (on the standardized features) selecting the 10 features with the largest between-class variance, including `word_freq_remove`, `word_freq_your`, `word_freq_000` and `char_freq_`.

The union of the two rankings gave 15 unique features, which were used for model training instead of all 57. This reduces the risk of overfitting and speeds up training without a meaningful drop in performance, since both tests agreed on most of the highly ranked attributes.

7 Machine Learning Algorithm

Logistic Regression was chosen as the primary supervised model.

Working principle: Logistic regression models the log-odds of the positive class as a linear combination of the input features and squashes the result through the sigmoid function to obtain a probability between 0 and 1. A threshold (0.5 by default) is applied to this probability to obtain the final class label.

Mathematical formulation:

$$P(y = 1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (1)$$

The weights w and bias b are learned by minimizing the binary cross-entropy loss:

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2)$$

Advantages: Fast to train, easy to interpret through its coefficients, and works well when classes are close to linearly separable, which is roughly true here after scaling.

Limitations: Assumes a linear decision boundary in the feature space, so it can underperform on datasets with strongly non-linear class boundaries, and it can be sensitive to outliers in the input features.

As a complementary unsupervised check, **K-Means clustering** ($k = 2$) was applied to the same selected features to see whether the natural grouping in the data aligns with the spam/non-spam labels without ever showing the model the labels.

8 Experimental Setup

Python Version	3.10
Scikit-Learn Version	1.3
IDE	Jupyter Notebook
Operating System	Ubuntu Linux
Hardware	Standard CPU, no GPU required

9 Hyperparameter Selection

Parameter	Value
Solver	lbfgs (default)
Regularization strength (C)	1.0
Max iterations	1000
Number of selected features (k)	10 per selection method
K-Means clusters	2
Random state	42

10 Results

Metric	Value
Accuracy	0.864
Precision	0.864
Recall	0.782
F1-score	0.821
ROC-AUC	0.948

For reference, the validation split gave an accuracy of 0.910 and an F1-score of 0.881, indicating the model generalizes reasonably from validation to the held-out test set. The K-Means clustering (unsupervised) achieved a silhouette score of 0.314 on the same 15 selected features, showing a moderate but not perfectly clean separation between the two natural clusters in the data.

11 Result Visualization

Confusion Matrix

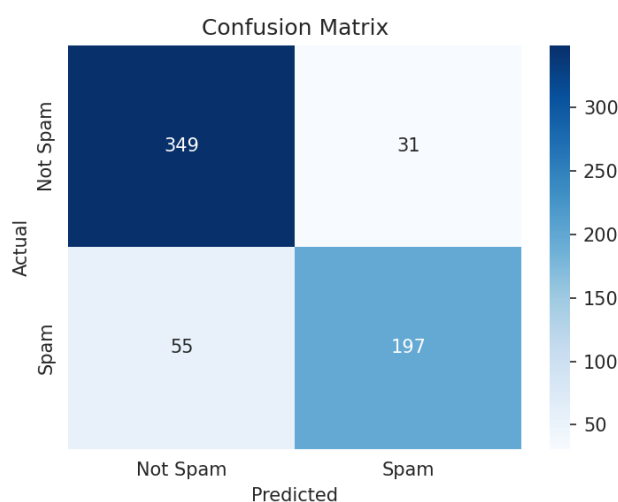


Figure 6: Confusion matrix on the test set

Inference: The model correctly classifies the majority of both classes, with more false negatives (spam predicted as not spam) than false positives. This matches the recall value of 0.782 being lower than the precision of 0.864. In a real deployment, this trade-off would need to be tuned depending on whether missing a spam email or blocking a genuine one is considered worse.

ROC Curve

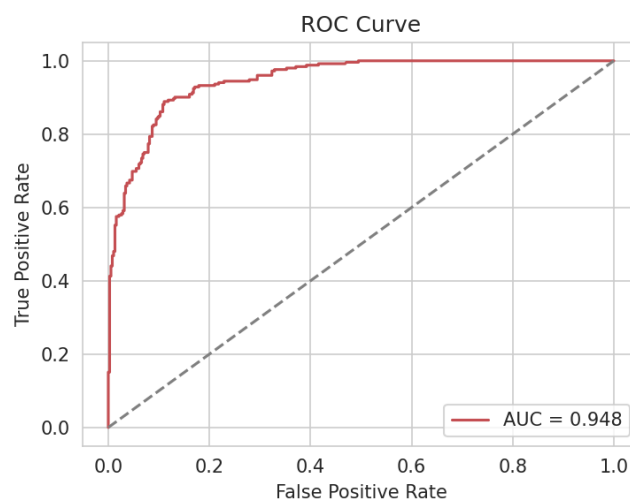


Figure 7: ROC curve for the logistic regression classifier

Inference: The curve stays well above the diagonal across all thresholds, and the area under the curve of 0.948 indicates strong separability between the two classes. This is a better summary of model quality than accuracy alone since it does not depend on a fixed decision threshold. It also suggests that adjusting the classification threshold could improve recall for spam detection at a small cost to precision, if that trade-off is preferred.

Precision-Recall Curve

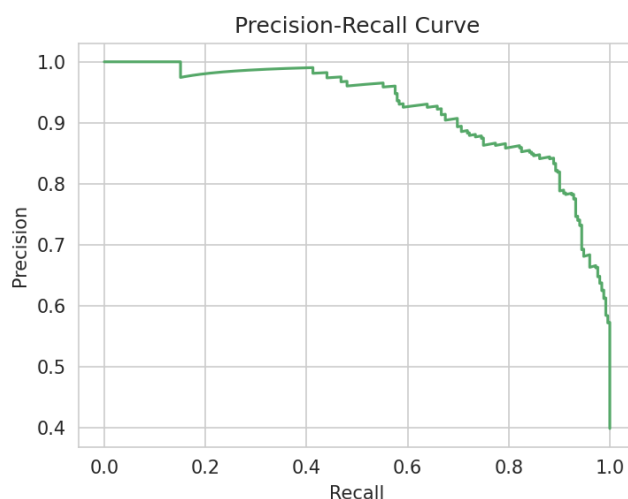


Figure 8: Precision-Recall curve for the logistic regression classifier

Inference: Precision stays high for a large range of recall values before dropping off sharply near the highest recall levels. This means the model can catch most spam emails while keeping false positives low, but pushing recall to its maximum comes at a steep precision cost. This curve is generally more informative than the ROC curve here since the classes are not perfectly balanced.

Feature Importance

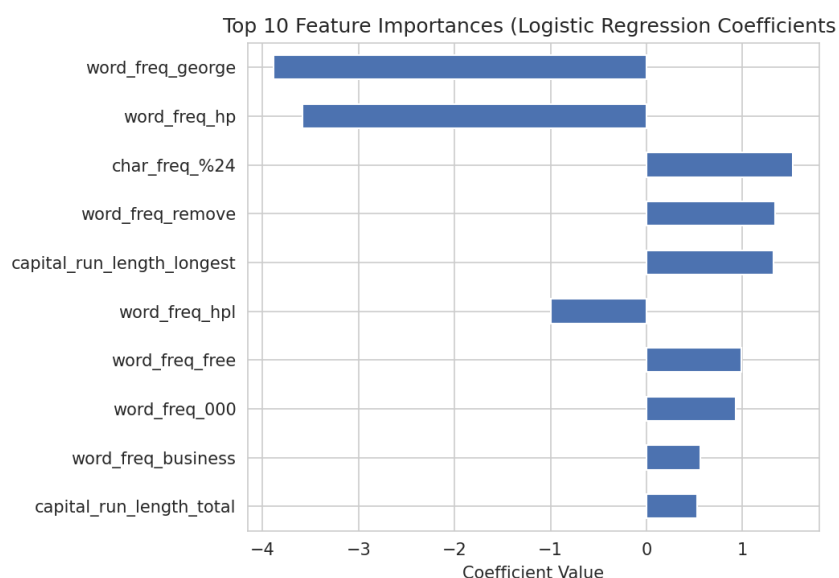


Figure 9: Top 10 features ranked by logistic regression coefficient magnitude

Inference: `word_freq_george` and `word_freq_hp` carry the largest negative coefficients, meaning their presence strongly pushes a prediction toward "not spam" – likely be-

cause these terms are tied to the personal/work email account used to collect the non-spam samples in this dataset. On the positive side, `char_freq_$`, `word_freq_remove` and `capital_run_length_longest` push predictions toward spam, matching the intuitive idea that dollar signs, the word "remove" (common in spam opt-out lines) and long runs of capital letters are typical spam markers.

12 Discussion

The logistic regression model performs well on this dataset, achieving an ROC-AUC above 0.94 and a test accuracy of 86.4% using only 15 of the original 57 features. Combining chi-square and ANOVA feature selection gave a compact and stable feature set, and both tests largely agreed on which attributes mattered most, which adds confidence that these features are genuinely informative rather than artifacts of one particular test. The main weakness observed is a recall of 0.782, meaning close to 22% of spam emails in the test set are missed; this could likely be improved by lowering the classification threshold or trying an ensemble method such as random forest or gradient boosting, at the cost of some interpretability. The unsupervised K-Means result (silhouette score 0.314) confirms that the two classes have some natural separation in feature space even without labels, though the moderate score shows the clusters are not as cleanly separated as a supervised model can achieve using label information directly.

13 Conclusion

Exploratory analysis showed clear differences between spam and non-spam emails in specific word frequencies and capitalization patterns, which was confirmed quantitatively through chi-square and ANOVA feature selection. After standardizing the data and reducing it to 15 relevant features, a logistic regression classifier achieved 86.4% accuracy and an ROC-AUC of 0.948 on the held-out test set, while a K-Means clustering baseline showed that even without labels, the data carries a moderate degree of natural class separation. Overall, the experiment demonstrates a complete pipeline from raw data to an interpretable classifier, and highlights the value of combining exploratory analysis with formal feature selection rather than relying on either alone.

14 References

-]
- [1] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase Data Set," UCI Machine Learning Repository, 1999.
 - [2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
 - [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
 - [4] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.